

Báo Cáo Phân Tích Hệ Thống URL Shortener Microservices

Môn học: SE361.Q21 — Kiến trúc và Thiết kế Phần mềm

Dự án: URL Shortener Microservices

Ngày: 13/07/2026

Báo Cáo Phân Tích Kiến Trúc Hệ Thống URL Shortener Microservices

Mục Lục

- Tổng Quan Kiến Trúc Microservices
- Domain-Driven Design & Bounded Context Analysis
- API Gateway - Thành Phần & Cơ Chế Hoạt Động
- Phân Tích Kiến Trúc Các Service
- Shared Packages
- Event-Driven Architecture & Outbox Pattern
- Infrastructure & Deployment
- Monitoring & Observability
- Các Thuật Toán & Cơ Chế Cốt Lõi
- Điểm Mạnh, Điểm Yếu & Khuyến Nghị
- Kết Luận
- Phụ Lục

1. Tổng Quan Kiến Trúc Microservices

1.1 Danh sách 5 Services

Hệ thống được thiết kế theo kiến trúc microservices với **5 services chính** và **3 shared packages**, được tổ chức trong một Go workspace monorepo (`go.work`):

#	Service	Vai trò	DB riêng	Port
1	Gateway (<code>gateway/</code>)	API Gateway, reverse proxy, authentication, rate limiting, circuit breaker, metrics	Redis (rate limit)	8080
2			PostgreSQL (<code>user_db</code>)	8083

#	Service	Vai trò	DB riêng	Port
	User Service (services/ user-service/)	Đăng ký, đăng nhập, quản lý người dùng		
3	URL Service (services/ url-service/)	Tạo, đọc, xóa short URL, redirect, caching	PostgreSQL (url_db) + Redis (cache)	8081
4	Analytics Service (services/analytics-service/)	Ghi nhận click, thống kê, milestone	PostgreSQL (analytics_db)	8082
5	Notification Service (services/ notification-service/)	Thông báo URL events cho người dùng	PostgreSQL (notification_db)	8084

Frontend là một SPA (Single Page Application) dựng bằng Vite + React (tại frontend/), giao tiếp với backend qua Gateway.

1.2 Tech Stack Chi Tiết

- **Ngôn ngữ:** Go 1.23.0 (Backend), TypeScript/React (Frontend)
- **Thư viện chính:** net/http (stdlib), ReverseProxy (stdlib), pgx/v5 (PostgreSQL), go-redis/v9 (Redis), amqp091-go (RabbitMQ), jwt/v5 (JWT), prometheus/client_golang (Metrics).
- **Cơ sở dữ liệu:** PostgreSQL 16 (Relational DBs), Redis 7 (Rate limit, Caching), RabbitMQ 3.13 (Message broker).
- **Hạ tầng:** Nginx 1.27 (Reverse Proxy đầu vào), Docker Compose (Local development), Kubernetes (Production deployment).
- **Giám sát:** Prometheus 2.53 (Metrics), Grafana 11.1 (Dashboards), Loki 2.9 + Promtail (Log aggregation).

1.3 Luồng Request Điển Hình

- **Luồng 1 (User Đăng Ký/Login):** Client → Nginx → Gateway (Bỏ qua Auth/Rate Limit) → User Service → PostgreSQL (user_db). Trả về JWT token khi đăng nhập thành công.
- **Luồng 2 (Tạo Short URL):** Client → Nginx → Gateway (JWT Verify → Rate Limit → Circuit Breaker) → URL Service (Auth Verify → DB Insert & Outbox Insert trong cùng 1 Transaction → Redis Cache) → Trả về Short Code. Background Outbox Coordinator sẽ poll event từ DB và publish lên RabbitMQ.

- **Luồng 3 (Redirect URL):** Client → Nginx → Gateway (Rate Limit → Circuit Breaker) → URL Service (Check Redis Cache -> HIT: Redirect ngay; MISS: Query PostgreSQL -> Cache write -> Redirect). Event click được đẩy bất đồng bộ vào Outbox để xử lý sau.

1.4 Sơ Đồ Kiến Trúc Tổng Thể



2. Domain-Driven Design & Bounded Context Analysis

2.1 Xác định 4 Bounded Contexts

Hệ thống được mô hình hóa rõ ràng thành **4 Bounded Contexts**:

Bounded Context 1: User & Authentication

- **Service & Database:** `user-service` / PostgreSQL (`user_db`)
- **Aggregate Root:** `User` (`id`, `email`, `password_hash`, `created_at`)
- **Value Objects:** `Password` (bcrypt-hashed), `Email` (validated), `JWT Token` (HMAC-SHA256 signed)
- **Repositories:** `UserRepository` interface (`pgxUserStore` implementation)

Bounded Context 2: URL Shortening

- **Service & Databases:** `url-service` / PostgreSQL (`url_db`) + Redis (cache)
- **Aggregate Root:** `ShortURL` (`id`, `shortCode`, `originalURL`, `userID`, `userEmail`, `expiresAt`, `isActive`)
- **Entities & Value Objects:** `URLRecord`, `OutboxRecord`, `ShortCode` (base62), `CachedURL`
- **Domain Events:** `URLCreatedEvent`, `URLClickedEvent`, `URLDeletedEvent` (published via Outbox)
- **Domain Services:** `URLService` (business logic), `ShortCodeGenerator`, `OutboxCoordinator`, `URLValidator`

Bounded Context 3: Analytics & Milestones

- **Service & Database:** `analytics-service` / PostgreSQL (`analytics_db`)
- **Aggregate Root:** `Click` (`shortCode`, `clickedAt`, `ipHash`, `userAgent`, `referer`)
- **Entities:** `ClickRecord`, `DeduplicationRecord` (`event_id`), `MilestoneRecord`
- **Domain Events:** Consumes `URLClickedEvent` → Emit `MilestoneReachedEvent` (khi đạt 10, 100, 1000 clicks)
- **Domain Services:** `MilestoneChecker`, `DeduplicationService` (đảm bảo idempotency)

Bounded Context 4: Notifications

- **Service & Database:** `notification-service` / PostgreSQL (`notification_db`)
- **Aggregate Root:** `Notification` (`id`, `userID`, `userEmail`, `eventType`, `payload`, `createdAt`)
- **Domain Events:** Consumes `URLCreatedEvent`, `URLDeletedEvent`, `MilestoneReachedEvent`

2.2 Context Mapping

Mối quan hệ giữa các Bounded Context được định nghĩa thông qua các pattern:

- **Open Host Service (OHS):** Tất cả services expose REST API qua Gateway.
- **Published Language / Event-Driven:** Giao tiếp bất đồng bộ thông qua RabbitMQ (Downstream services consume events từ topic exchange).

- **Separate Ways:** Dữ liệu hoàn toàn độc lập, mỗi service sở hữu DB riêng.
- **Anti-Corruption Layer (ACL):** Gateway đóng vai trò làm ACL để bảo vệ hệ thống nội bộ (Authentication, Rate Limiting, CORS, Circuit Breaker).

2.3 Event Storming - Domain Events

Event Type	Producer	Consumer(s)	Trigger	Payload
url.created	URL Service	Analytics, Notification	Tạo short URL thành công	short_code, original_url, user_id, user_email, expires_at
url.clicked	URL Service	Analytics	Click vào short URL	short_code, user_id, user_email, ip_hash, user_agent, referer, clicked_at
url.deleted	URL Service	Notification	Xóa/Hủy kích hoạt short URL	short_code, user_id, user_email
milestone.reached	Analytics Service	Notification	Đạt ngưỡng click (10/100/1000)	short_code, user_id, user_email, milestone, total_clicks

Sơ đồ Luồng Sự Kiện (Event Flow Diagram - Event Storming)

```
flowchart TD
    %% Styling Definitions
    classDef actor fill:#FFE5CC,stroke:#F4B084,stroke-width:2px;
    classDef command fill:#DDEBF7,stroke:#9BC2E6,stroke-width:2px;
    classDef aggregate fill:#FFF2CC,stroke:#FFD966,stroke-width:2px;
    classDef event fill:#FCE4D6,stroke:#F8CBAD,stroke-width:2px;
    classDef service fill:#E2F0D9,stroke:#A9D08E,stroke-width:2px;

    %% 1. Create Short URL Flow
    subgraph CreateShortURLFlow [1. Create Short URL Flow]
        User1([Authenticated User]):::actor --> CmdCreate[Command: Create Short URL]:::command
        CmdCreate --> Agg1[Aggregate: ShortURL]:::aggregate
        Agg1 -->|1. Register & Validate<br/>2. Generate Code<br/>3. Store URL & Outbox| EvtCreated[Event: url.created]:::event
        EvtCreated --> NotifSvc1[Notification Service]:::service
        EvtCreated --> AnalSvc1[Analytics Service]:::service
    end

    %% 2. Click Short URL Flow
    subgraph ClickShortURLFlow [2. Click Short URL Flow]
        User2([Anonymous User]):::actor --> CmdClick[Command: Click Short URL]:::command
        CmdClick --> Agg2[Aggregate: ShortURL]:::aggregate
        Agg2 -->|1. Check Cache<br/>2. Query DB<br/>3. Redirect 308|
```

```

EvtClicked[Event: url.clicked] :: event
  EvtClicked → AnalSvc2[Analytics Service] :: service
  AnalSvc2 → |1. Deduplicate<br/>2. Insert Click<br/>3. Check
Threshold| EvtMilestone[Event: milestone.reached] :: event
  EvtMilestone → NotifSvc2[Notification Service] :: service
end

%% 3. Delete Short URL Flow
subgraph DeleteShortURLFlow [3. Delete Short URL Flow]
  User3([Owner / Auth User]) :: actor → CmdDelete[Command: Delete Short
URL] :: command
  CmdDelete → Agg3[Aggregate: ShortURL] :: aggregate
  Agg3 → |1. Deactivate URL<br/>2. Invalidate Cache| EvtDeleted[Event:
url.deleted] :: event
  EvtDeleted → NotifSvc3[Notification Service] :: service
end

%% Force vertical layout mapping
CreateShortURLFlow ~~~ ClickShortURLFlow
ClickShortURLFlow ~~~ DeleteShortURLFlow

```

2.4 Aggregate Roots

User Aggregate (User Context)

- **Schema:** User { id: string (UUID) email: Email (Value Object) passwordHash: string (bcrypt) createdAt: timestamp }
- **Invariants:** Email phải là độc nhất (unique), password đã được mã hóa bằng thuật toán an toàn.
- **Repository:** UserRepository.

ShortURL Aggregate (URL Context)

- **Schema:** ShortURL { id: string (UUID) shortCode: ShortCode (7-char base62) originalURL: URL (Value Object) userID: string userEmail: string createdAt: timestamp expiresAt: timestamp | nil isActive: boolean }
- **Invariants:** shortCode phải độc nhất, originalURL phải tuân thủ đúng định dạng scheme (http/https).
- **Business Rules:**
 - Không chuyển hướng nếu URL đã quá hạn hoặc đã bị hủy kích hoạt (isActive = false).
 - Chỉ người sở hữu URL (owner) mới có quyền hủy kích hoạt URL đó.
- **Repository:** URLStore.
- **Caching:** Cache interface cho Redis projection.

Click Aggregate (Analytics Context)

- **Schema:** `Click { shortCode: string clickedAt: timestamp ipHash: string (one-way hash) userAgent: string referer: string }`
- **Invariants:** Đảm bảo idempotency bằng cách kiểm tra trùng lặp thông qua `event_id` trước khi insert click record.
- **Repository:** `ClickRepository`.

Notification Aggregate (Notification Context)

- **Schema:** `Notification { id: string (UUID) userID: string userEmail: string eventType: string payload: json createdAt: timestamp }`
- **Repository:** `NotificationRepository`.

2.5 Ubiquitous Language

Thuật ngữ	Định nghĩa	Context
Short Code	Mã 7 ký tự Base62 đại diện cho URL gốc.	URL
Short URL	URL đầy đủ dạng <code>http://host/r/{short_code}</code> .	URL
Original URL	URL đích dài cần rút gọn.	URL
Redirect	Chuyển hướng HTTP 308 từ Short URL sang Original URL.	URL
Click	Một lần truy cập thực tế vào Short URL của người dùng.	Analytics
Milestone	Ngưỡng click đạt cột mốc (10, 100, 1000 clicks).	Analytics
Outbox	Bảng database tạm thời lưu trữ các domain events chờ publish.	URL
Deduplication	Cơ chế lọc trùng lặp event để đảm bảo tính idempotent.	Analytics
Hash IP	Băm một chiều địa chỉ IP của client để bảo vệ quyền riêng tư.	URL, Analytics
Rate Limit	Giới hạn số lượng request trong một khoảng thời gian xác định.	Gateway
Circuit Breaker	Cơ chế ngắt kết nối tạm thời tới upstream service khi có lỗi.	Gateway

2.6 Quyết định Thiết kế Chiến lược (Strategic Design Decisions)

1. **Tách biệt Database hoàn toàn:** Mỗi service sở hữu PostgreSQL riêng. Đảm bảo loose coupling tối đa và cho phép schema tiến hóa độc lập.

- 2. **Giao tiếp hướng sự kiện (Event-Driven):** Sử dụng RabbitMQ với Topic Exchange pattern. URL Service publish events, các downstream services consume bất đồng bộ, tránh làm chậm luồng chính.
- 3. **Transactional Outbox Pattern:** Đảm bảo tính nhất quán (eventual consistency) giữa trạng thái database và tin nhắn gửi đi mà không cần dùng distributed transactions.
- 4. **API Gateway làm Anti-Corruption Layer (ACL):** Tập trung hóa authentication, rate limiting, và circuit breaker tại Gateway để giải phóng các service nội bộ khỏi các nghiệp vụ phụ trợ này.
- 5. **Double JWT Verification:** Cả Gateway và internal services đều verify JWT nhằm gia tăng tính bảo mật (Defense in Depth).
- 6. **Cache-aside Pattern:** Sử dụng Redis cache kèm TTL hợp lý để giảm tải cho PostgreSQL của URL Service trong luồng redirect.

3. API Gateway - Thành Phần & Cơ Chế Hoạt Động

API Gateway là cửa ngõ duy nhất của hệ thống, chịu trách nhiệm cho các vấn đề cross-cutting concerns:

3.1 Cơ Chế Cấu Hình

Cấu hình hoàn toàn qua biến môi trường (Environment Variables). Hệ thống sẽ dừng hoạt động ngay lập tức (fail-fast) nếu thiếu các cấu hình bắt buộc.

3.2 Cơ Chế Routing & Path Rewriting

Gateway sử dụng cơ chế so khớp tiền tố đường dẫn (First-match prefix matching) để phân phối requests tới các upstream services phù hợp:

Method	PathPrefix	Upstream Service	Auth	Rate Limit Key	Strip Prefix
POST	/api/auth/register	user-service	×	-	/api/auth
POST	/api/auth/login	user-service	×	-	/api/auth
GET	/api/me	user-service	✓	-	/api
POST	/api/shorten	url-service	✓	shorten	/api

Method	PathPrefix	Upstream Service	Auth	Rate Limit Key	Strip Prefix
GET	/api/urls	url-service	✓	-	/api
DELETE	/api/urls/	url-service	✓	-	/api
GET	/r/	url-service	×	redirect	/r
GET	/api/stats/	analytics-service	×	-	/api
GET	/api/notifications	notification-service	✓	-	/api

Phân tích quyết định định tuyến (Routing Decisions):

- **Các endpoints xác thực (Auth) không giới hạn tần suất (Rate limit):** Đăng ký (/register) và Đăng nhập (/login) hiện tại không bị giới hạn rate limit. Đây là một rủi ro bảo mật lớn (dễ bị tấn công brute-force mật khẩu). Cần đề xuất bổ sung rate limit riêng cho các endpoints này.
- **Endpoint rút gọn URL (/shorten) giới hạn 10 req/60s:** Hợp lý để ngăn ngừa hành vi spam tạo URL rác làm tràn ngập dữ liệu.
- **Endpoint redirect /r/ giới hạn cao (300 req/60s):** Phục vụ lượng truy cập lớn của người dùng phổ thông, đảm bảo tính sẵn sàng cao của dịch vụ chính.
- **Endpoint thống kê (/stats/) công khai:** Bất kỳ người dùng ẩn danh nào cũng có thể xem thống kê của URL nếu biết short code. Điều này phù hợp với mô hình chia sẻ thông số công khai của nhiều nền tảng rút gọn URL.
- **Endpoint Notifications yêu cầu JWT ở cả 2 đầu:** Chỉ người dùng sở hữu tài khoản mới xem được thông báo tương ứng của mình, đảm bảo tính riêng tư dữ liệu.

3.3 Reverse Proxy Implementation

Sử dụng `net/http/httputil.ReverseProxy` của Go Standard Library. Đường dẫn gốc được viết lại (strip prefix) và lưu vào request context trước khi proxy chuyển tiếp. Hỗ trợ forward các header chuẩn: `X-Forwarded-For`, `X-Real-IP`, và `X-Correlation-ID`. - **Điểm mạnh (Strengths):** Tận dụng tối đa thư viện chuẩn của Go để tối ưu hiệu năng (zero allocation hot path); viết lại đường dẫn linh hoạt; hỗ trợ forward header chuẩn. - **Điểm yếu (Weaknesses):** Thiếu cơ chế tự động thử lại (retry logic) khi upstream lỗi; cấu hình hồ chứa kết nối (connection pooling) mặc định; thiếu cấu hình timeout độc lập cho từng service.

3.4 JWT Authentication Middleware

Trước khi chuyển tiếp request đến các route yêu cầu xác thực (`RequiresAuth: true`), Middleware sẽ: 1. Trích xuất Bearer token từ `Authorization` header. 2. Kiểm tra chữ ký số JWT bằng `JWT_SECRET`. 3. Giải mã và lưu thông tin `Claims` vào request context cho các handler tiếp theo sử dụng. - **Lý do thiết kế:** Đóng vai trò là lớp kiểm soát truy cập vòng ngoài (ACL). Giúp lọc bỏ các request không hợp lệ sớm nhất có thể trước khi chạm vào các service nghiệp vụ bên trong.

3.5 Request Handler & Health Check

Đóng vai trò điều phối chính cho request đi qua Gateway. - **Quy trình xử lý (Request Flow):** So khớp Route -> Check Auth (JWT) -> Check Rate Limiting -> Strip Prefix -> Check Circuit Breaker (cho url-service) -> Thực hiện Proxy -> Ghi nhận Prometheus Metrics. - **Thiết kế:** Sử dụng Dependency Injection (DI) để truyền đối tượng qua các interfaces (`rateLimiter`, `circuitBreaker`), hỗ trợ cô lập và dễ viết unit test. - **Tối ưu & An toàn:** Bản tin phản hồi JSON / `health` được pre-encoded khi khởi động để tối ưu hiệu năng. Sử dụng shallow check (không ping database) để tránh gây quá tải DB.

3.6 Rate Limiting & Cơ Chế Fail-Open

- **Thuật toán:** Fixed Window, lưu trữ counters trong Redis với thời gian timeout cho mỗi thao tác là 100ms.
- **Key format:** `rl:{route_key}:{client_ip}` (ví dụ: `rl:shorten:192.168.1.1`).
- **Thiết kế Fail-Open:** Nếu kết nối Redis gặp sự cố, hệ thống chỉ ghi log cảnh báo và **vẫn cho phép request đi qua** nhằm đảm bảo tính sẵn sàng tối đa của dịch vụ (Availability > Consistency).
- **Điểm yếu:** Thuật toán Fixed Window dễ bị vượt ngưỡng giới hạn ở biên của 2 cửa sổ thời gian (Boundary problem); phụ thuộc vào Redis (SPOF); dễ bị vượt qua bằng cách đổi IP (nếu không giới hạn theo User ID) hoặc giả mạo header `X-Forwarded-For`.

3.7 Circuit Breaker State Machine

Bảo vệ Gateway khỏi tình trạng cascading failure khi URL Service gặp sự cố:

```
stateDiagram-v2
    [*] --> CLOSED

    CLOSED --> OPEN : failures ≥ maxFailures\n(trong failureWindow)

    OPEN --> HALF_OPEN : openTimeout expired\n(next request arrives as probe)
```

```
HALF_OPEN → CLOSED : success (probe request succeeded)
HALF_OPEN → OPEN : failure (probe request failed, reset timer)
```

- **CLOSED:** Requests được chuyển tiếp bình thường.
- **OPEN:** Requests bị reject ngay lập tức tại Gateway với HTTP 503 Service Unavailable mà không gọi upstream.
- **HALF-OPEN:** Khi hết timeout, cho phép **duy nhất 1 request** đi qua để kiểm tra (probe). Thành công → CLOSED; Thất bại → OPEN trở lại.
- **Điểm mạnh:** Hỗ trợ lock-free tối đa cho hầu hết các hoạt động đọc trạng thái; tích hợp kiểm tra hủy bỏ context (`context cancellation`); cơ chế Half-Open probe chỉ cho phép 1 request đi qua để thử nghiệm nhằm bảo vệ hệ thống triệt để.
- **Điểm yếu:** CB không có bộ định thời phục hồi tự động (chỉ chuyển từ Open sang Half-open khi có request mới tới Gateway); chỉ đang cấu hình bảo vệ cho duy nhất `url-service` ; coi tất cả mọi loại lỗi (gồm cả lỗi từ client 4xx hoặc network timeout) là lỗi hệ thống để tăng counter.

3.8 Correlation ID & CORS Middleware

- **Correlation ID:** Trích xuất hoặc tự sinh UUID mới thông qua header `X-Correlation-ID` và inject vào ngữ cảnh request context. Giúp trace log phân tán một cách đồng bộ xuyên suốt qua toàn bộ hệ thống các service nội bộ.
- **CORS:** Thiết lập CORS mở rộng cho môi trường local (`Access-Control-Allow-Origin: *`).
- **Điểm yếu:** Sử dụng wildcard Origin `*` là một lỗ hổng bảo mật nếu deploy trực tiếp lên production. Cần cấu hình động dựa trên domain cụ thể.

4. Phân Tích Kiến Trúc Các Service

Mỗi microservice tuân thủ kiến trúc phân tầng (Handlers/Controllers → Business Services → Repositories/Stores) nhằm đảm bảo tính độc lập và khả năng viết unit test dễ dàng:

4.1 User Service

- **Nhiệm vụ:** Quản lý định danh và phiên đăng nhập.
- **Cơ chế bảo mật nổi bật:**
- **Bảo vệ tấn công Timing Attack:** Khi user không tồn tại trong DB, hệ thống vẫn thực hiện so khớp password với một "dummy hash" ngẫu nhiên để tránh việc kẻ tấn công dò tìm email tồn tại dựa trên thời gian phản hồi.

- Sử dụng Bcrypt (cost=12) để băm mật khẩu.

4.2 URL Service

- **Nhiệm vụ:** Rút gọn URL, điều hướng redirect và quản lý vòng đời short code.
- **Cơ chế lưu kho (Caching):** Cache-aside pattern với Redis. Endpoint redirect check Redis trước (timeout 50ms), nếu miss mới query DB và lưu lại vào cache bất đồng bộ.
- **Bảo toàn Event:** Sử dụng transactional outbox pattern để đảm bảo việc tạo URL và ghi nhận event tạo mới luôn diễn ra atomic (cùng thành công hoặc cùng thất bại).

4.3 Analytics Service

- **Nhiệm vụ:** Thống kê click chuột, tổng hợp dữ liệu theo timeline và phát hiện Milestone.
- **Xử lý idempotent:** Đảm bảo mỗi event click từ RabbitMQ chỉ được xử lý một lần thông qua bảng đối chiếu deduplication (`event_id` làm khóa chính).

4.4 Notification Service

- **Nhiệm vụ:** Lắng nghe các event từ hệ thống và tạo thông báo tương ứng cho người dùng.
- **Cơ chế hoạt động:** Sử dụng Manual Acknowledgement (ACK) khi consume tin nhắn từ RabbitMQ. Chỉ ACK khi đã lưu thông báo vào DB thành công; nếu lỗi sẽ gửi NACK kèm cơ chế Requeue để xử lý lại.

5. Shared Packages

Các thư viện dùng chung được định nghĩa độc lập để tránh trùng lặp code:

1. **shared/auth** : Chứa logic sinh/verify JWT Token, cấu hình Middleware kiểm tra xác thực và các helper trích xuất claims từ context.
 2. **shared/events** : Định nghĩa cấu trúc chuẩn của các Domain Events (`BaseEvent` , `URLCreatedEvent` , `URLClickedEvent` , `URLDeletedEvent` , `MilestoneReachedEvent`).
 3. **shared/logger** : Logger cấu trúc sử dụng thư viện chuẩn `log/slog` định dạng JSON. Tự động trích xuất Correlation ID từ context và ghi nhận thông tin HTTP request.
-

6. Event-Driven Architecture & Outbox Pattern

6.1 Transactional Outbox Pattern

Nhằm giải quyết bài toán phân tán dữ liệu mà không cần dùng Distributed Transactions (như 2PC), hệ thống áp dụng Transactional Outbox Pattern:

```
sequenceDiagram
    autonumber
    actor Client
    participant URLSvc as URL Service
    participant DB as PostgreSQL (url_db)
    participant OutboxCoord as Outbox Coordinator
    participant RabbitMQ as RabbitMQ Broker

    Client->>URLSvc: POST /api/shorten

    rect rgb(240, 248, 255)
        Note over URLSvc, DB: Local Transaction (Atomic)
        URLSvc->>DB: 1. INSERT INTO urls (...)
        URLSvc->>DB: 2. INSERT INTO outbox (event_type, payload)
        URLSvc->>DB: COMMIT TRANSACTION
    end

    URLSvc-->>Client: 201 Created (short_code, ...)

    loop Mỗi 2 giây (Background Polling)
        OutboxCoord->>DB: SELECT FOR UPDATE SKIP LOCKED (LIMIT 50)
        DB-->>OutboxCoord: Trả về các events chưa gửi
        Note over OutboxCoord: Khóa events (locked_until = now + 30s)
        OutboxCoord->>RabbitMQ: Publish events (Topic Exchange)
        OutboxCoord->>DB: UPDATE outbox SET published_at = now()
    end
```

Cơ chế này đảm bảo thuộc tính **At-Least-Once Delivery** – tin nhắn chắc chắn sẽ được gửi đi, dù cho broker RabbitMQ có bị down tại thời điểm tạo URL.

6.2 RabbitMQ Topic Exchange

Hệ thống sử dụng Topic Exchange (`url-shortener`) để định tuyến tin nhắn linh hoạt:

- `url.created` → Gửi đến Queue của Notification Service.
 - `url.clicked` → Gửi đến Queue của Analytics Service.
 - `url.deleted` → Gửi đến Queue của Notification Service.
 - `milestone.reached` → Gửi đến Queue của Notification Service.
-

7. Infrastructure & Deployment

7.1 Nginx Reverse Proxy

Nginx đứng ở ngoài cùng làm cổng tiếp nhận traffic HTTP (port 80):

- Prefix `/api/` và `/health` → Chuyển tiếp tới Upstream API Gateway.
- Prefix `/r/` (Redirect) → Chuyển tiếp tới Upstream API Gateway.
- Các path khác `/` → Phục vụ giao diện tĩnh của Frontend.

7.2 Docker Compose Topology

Trong môi trường phát triển local, Docker Compose quản lý **18 containers**:

- 4 Database PostgreSQL riêng biệt.
- 1 Redis instance + 1 RabbitMQ broker.
- 5 backend services (Gateway + 4 services).
- Frontend SPA server.
- Monitoring stack (Prometheus, Grafana, Loki, Promtail, Adminer). Tất cả container được cấu hình Healthcheck rõ ràng nhằm đảm bảo tự khởi động (ví dụ: DB khởi động xong thì Service mới chạy).

7.3 Kubernetes Manifests

Mô tả kiến trúc triển khai trên Production:

- **url-service** : Scale lên 3 bản sao (Replicas) để đáp ứng lượng traffic redirect lớn.
 - **gateway** : Scale lên 2 bản sao đảm bảo High Availability (HA), expose qua NodePort.
 - **ConfigMap & Secrets**: Dùng `app-config` cho các chuỗi kết nối và `JWT_SECRET` được bảo mật qua Secret.
-

8. Monitoring & Observability

Hệ thống được tích hợp sẵn giải pháp quan sát toàn diện:

- **Prometheus**: Thu thập metrics định kỳ (scrape_interval: 5s) từ `/metrics` endpoint của tất cả 5 services. Gateway export metrics về trạng thái Circuit Breaker, số lượng requests, và biểu đồ phân bố latency.

- **Grafana:** Cấu hình sẵn 2 dashboards (`circuit_breaker.json` và `services_overview.json`) hiển thị trực quan sức khỏe hệ thống.
 - **Loki + Promtail:** Gom log tập trung từ các container và hiển thị trên Grafana.
-

9. Các Thuật Toán & Cơ Chế Cốt Lõi

9.1 Base62 Codec (Thuật toán rút gọn URL)

Để tạo short code ngắn gồm 7 ký tự từ một URL gốc:

1. Hệ thống sinh số ngẫu nhiên lớn bằng `crypto/rand` (đảm bảo tính ngẫu nhiên an toàn cao hơn `math/rand`).
2. Sử dụng thuật toán Base62 (`math/big`) để mã hóa số ngẫu nhiên này thành chuỗi ký tự chứa `[a-zA-Z0-9]` .
3. Số lượng mã tối đa: $62^7 \approx 3.52$ nghìn tỷ mã. Xác suất va chạm cực kỳ thấp. Hệ thống hỗ trợ retry 3 lần nếu có va chạm code trong DB.

9.2 Outbox Coordinator Worker Pool

Để publish events hiệu quả mà không gây nghẽn:

- Một Goroutine chính thực hiện poll bảng `outbox` mỗi 2 giây, lấy tối đa 50 bản ghi chưa publish.
- Sử dụng truy vấn SQL `FOR UPDATE SKIP LOCKED` để các worker chạy song song không tranh chấp dữ liệu.
- Phân phối các công việc này cho 3 Worker Goroutines chạy ngầm thực hiện gửi message sang RabbitMQ và cập nhật DB.

9.3 Khóa mịn trong Circuit Breaker (Fine-grained Locking)

Để tránh deadlock và không chặn request khi ghi nhận metrics:

- Sử dụng `sync.Mutex` để đồng bộ trạng thái state machine của CB.
 - Hàm callback thông báo đổi trạng thái (`notifyStateChange`) được gọi **sau khi đã unlock** mutex để đảm bảo các tiến trình khác có thể tiếp tục truy cập CB ngay lập tức.
-

10. Điểm Mạnh, Điểm Yếu & Khuyến Nghị

10.1 Điểm Mạnh (Strengths)

1. **Thiết kế chuẩn Microservices:** Tách biệt DB hoàn toàn, loose coupling tốt, giao tiếp qua message broker.
2. **Độ tin cậy cao (Resiliency):**
 - Tích hợp Circuit Breaker ngăn lỗi lan truyền (cascading failure).
 - Áp dụng Outbox Pattern giúp chống mất mát dữ liệu sự kiện.
 - Thiết kế Rate Limit kiểu Fail-open bảo vệ dịch vụ.
3. **DDD Rõ Ràng:** Phân chia bounded contexts, aggregate roots và ubiquitous language nhất quán.
4. **Bảo mật:** Cơ chế dummy hash chống timing attack, mã hóa IP người dùng, bảo mật JWT hai lớp.
5. **DevOps-ready:** Cấu hình Kubernetes hoàn chỉnh, tích hợp đầy đủ Prometheus, Grafana, Loki.

10.2 Điểm Yếu (Weaknesses)

1. **Hạn chế của CB:** Chỉ mới áp dụng bảo vệ cho `url-service`, các service khác (User, Notification) chưa được cấu hình Circuit Breaker.
2. **Thuật toán Rate Limit đơn giản:** Sử dụng Fixed Window dễ gặp vấn đề quá tải đột biến tại biên giới hạn (Boundary problem).
3. **Cấu hình CORS quá thoáng:** Dùng `Access-Control-Allow-Origin: *` không an toàn cho production.
4. **Thiếu cơ chế tự động phục hồi của CB:** Circuit Breaker chỉ chuyển từ Open sang Half-Open khi có request mới tới (chứ không tự động phục hồi chủ động theo timer).
5. **Bảo mật:** Login endpoint chưa được cấu hình rate limit chống tấn công brute-force.
6. **Thiếu Distributed Tracing:** Gây khó khăn khi trace luồng lỗi giữa các service phân tán trong production.

10.3 Khuyến Nghị Cải Thiện (Recommendations)

Ưu tiên 1 (Cần làm ngay)

- **Thêm Circuit Breaker cho toàn bộ Upstream:** Đảm bảo tất cả các service gọi qua Gateway đều được bảo vệ.
- **Rate Limit cho Auth Endpoints:** Đặt giới hạn chặt chẽ cho các API `/register` và `/login`.

- **Sửa cấu hình CORS:** Thay thế wildcard * bằng danh sách tên miền được cho phép cụ thể qua env vars.
- **Thêm Timeout Graceful Shutdown:** Thiết lập timeout (ví dụ: 30s) cho hàm `Shutdown` của Gateway để tránh bị treo tiến trình vĩnh viễn khi có kết nối bị kẹt.

Ưu tiên 2 (High Priority)

- **Chuyển sang Sliding Window Rate Limit:** Để khắc phục nhược điểm biên của Fixed Window.
- **Tích hợp Distributed Tracing (Jaeger):** Kết hợp với Correlation ID có sẵn để vẽ bản đồ cuộc gọi phân tán.
- **Bổ sung Sentry:** Để bắt lỗi và cảnh báo ngoại lệ tập trung.

11. Kết Luận

Hệ thống URL Shortener Microservices là một mô hình thiết kế chuẩn chỉ và hoàn chỉnh bằng ngôn ngữ Go. Các thành phần được cấu trúc khoa học, tuân thủ các best practices về cloud-native,DDD và tính sẵn sàng cao. Bằng cách áp dụng một vài cải tiến nhỏ về mặt bảo mật (CORS, Rate Limit Auth) và độ tin cậy (CB cho toàn bộ services, Graceful Shutdown timeout), hệ thống hoàn toàn đáp ứng tốt các yêu cầu vận hành thực tế ở quy mô sản xuất.

Phụ Lục

Phụ Lục A: Các Quyết Định Kiến Trúc Chính

- **A.1 Sử dụng ReverseProxy của stdlib:** Tránh phụ thuộc thư viện ngoài, tối ưu hiệu năng, dễ bảo trì. Đánh đổi: Phải tự xây dựng Circuit Breaker và cơ chế Load Balancing.
- **A.2 Sử dụng RabbitMQ thay vì Kafka:** Do nhu cầu hệ thống là Event-Driven xử lý nghiệp vụ thông thường (chứ không phải Event Streaming khối lượng lớn), RabbitMQ cung cấp cơ chế định tuyến (Topic Exchange) linh hoạt và dễ vận hành hơn.
- **A.3 Cơ chế Fail-open của Rate Limiter:** Hệ thống coi Redis Rate Limiter là thành phần bảo vệ bổ sung. Nếu Redis gặp sự cố, hệ thống chọn hy sinh tính năng giới hạn để giữ cho dịch vụ hoạt động bình thường, thay vì chặn đứng toàn bộ người dùng.
- **A.4 Xác thực JWT hai lớp:** Gateway verify JWT để chặn sớm request xấu, nhưng các internal service vẫn thực hiện verify lại nhằm đảm bảo an toàn tuyệt đối phòng trường hợp mạng nội bộ bị xâm nhập hoặc request gọi trực tiếp bypass Gateway.

Phụ Lục B: Phân Tích Bảo Mật (Security)

Luồng Authorized Request Flow



Bảng Đánh Giá Mối Đe Dọa (Security Threat Assessment)

Mối đe dọa	Mức độ	Biện pháp giảm thiểu hiện tại
Đánh cắp token JWT	Cao	Đặt TTL ngắn (24h), khuyến nghị HTTPS trên production.
Tấn công Brute force Login	Trung bình	Mã hóa Bcrypt cost=12 (chậm), cơ chế dummy hash chống timing attack.
SQL Injection	Cao	Sử dụng pgx với parameterized queries (truy vấn tham số hóa).
CSRF	Trung bình	Token JWT truyền qua Authorization header thay vì Cookie.
Bypass Rate limit	Thấp	Rate limit dựa trên IP người dùng kết hợp fail-open.

Phụ Lục C: Phân Tích Hiệu Năng (Performance)

- **Xác suất va chạm Short Code:** Với độ dài 7 ký tự Base62 ($62^7 \approx 3.52 \times 10^12$ nghìn tỷ kết hợp), áp dụng nghịch lý ngày sinh nhật (Birthday paradox), sau khi tạo 1 triệu URLs, xác suất va chạm chỉ khoảng 1.4×10^{-7} \$. Với cơ chế sinh lại tối đa 3 lần, khả năng thất bại do va chạm là hoàn toàn không đáng kể (2.7×10^{-21} \$).

- **Hiệu năng cache:** Redis cache check được cài đặt timeout chặt chẽ ở mức 50ms. Nếu Redis phản hồi chậm, hệ thống tự động fallback sang database để tránh gây nghẽn cục bộ cho client. Tỷ lệ cache hit kỳ vọng cho flow redirect (đọc nhiều hơn ghi) là trên 90%.

Phụ Lục D: Phân Tích Cấu Trúc File & Dòng Code (Summary)

Thống kê quy mô mã nguồn

Component	Số lượng Files	Tổng số dòng code	Vai trò chính
Gateway	12	~1,000	Routing, Auth, Rate limit, Circuit Breaker
URL Service	16	~1,500	Quản lý URL, Redirect, Cache-aside, Outbox coordinator
User Service	11	~600	Quản lý người dùng, Xác thực mật khẩu Bcrypt
Shared Auth	2	~180	JWT Verify, Auth Middleware dùng chung
Shared Logger	1	~80	Structured logger slog JSON + Correlation ID

Báo cáo hoàn chỉnh. Hết tài liệu.

Phân Tích Chi Tiết URL Service — Microservice Rút Gọn URL

Mục Lục

- Tổng Quan URL Service
- Cấu Hình & Kết Nối Hạ Tầng
- Thiết Kế Cơ Sở Dữ Liệu (Schema Database)
- Các Hợp Phần Tiện Ích & Helper
- Tầng Lưu Trữ & Hạ Tầng Dữ Liệu
- Tầng Xử Lý Logic & Bootstrapping
- Bản Đồ Luồng Hoạt Động & Hiệu Năng
- Phân Tích Phi Chức Năng (Bảo Mật, Tin Cậy, Mở Rộng)
- Tích Hợp Shared Packages
- Kết Luận

1. Tổng Quan URL Service

URL Service là microservice chịu trách nhiệm quản lý vòng đời của liên kết rút gọn trong hệ thống `url-shortener-microservices`. Chi tiết về kiến trúc tổng thể của toàn hệ thống và cách URL Service kết nối với các microservices khác đã được mô tả tại [Báo cáo Tổng quan Kiến trúc](#).

Tài liệu này tập trung đi sâu phân tích cấu trúc thiết kế chi tiết bên trong của riêng URL Service.

1.1. Nguyên Lý Thiết Kế Cốt Lõi

Hệ thống được vận hành dựa trên 5 nguyên tắc thiết kế then chốt: 1. **Transactional Outbox**: Ghi nhận sự kiện nghiệp vụ vào database trong cùng transaction với thực thể chính, bảo đảm độ tin cậy truyền tin (At-Least-Once Delivery). 2. **Cache-Aside L1**: Tối ưu tốc độ điều hướng (Redirect) bằng cách lưu đệm thông tin URL active lên Redis trước khi truy vấn PostgreSQL. 3. **Fail-Open (Redis)**: Lớp Cache đệm không nằm trên critical path bắt buộc. Khi Redis gặp sự cố, hệ thống tự động bypass qua database để tránh gián đoạn dịch vụ. 4. **Cryptographic Randomness**: Loại bỏ rủi ro bị

tấn công dò tìm liên kết bằng cách sử dụng nguồn sinh số ngẫu nhiên an toàn của hệ điều hành (`crypto/rand`). 5. **Graceful Shutdown**: Thu hồi context để dừng background worker và chờ đóng kết nối HTTP Server an toàn (timeout 10 giây) khi nhận tín hiệu ngắt từ hệ điều hành.

2. Cấu Hình & Kết Nối Hạ Tầng

2.1. Cấu Hình Hệ Thống

Cấu trúc cấu hình hệ thống được định nghĩa qua đối tượng `Config` với các tham số chính sau:

Tham số	Trạng thái	Mặc định	Ý nghĩa & Đánh giá
<code>DatabaseURL</code>	Bắt buộc	—	Chuỗi kết nối PostgreSQL (DSN format).
<code>RedisURL</code>	Bắt buộc	—	Chuỗi kết nối Redis (DSN format).
<code>RabbitMQURL</code>	Bắt buộc	—	Chuỗi kết nối RabbitMQ (AMQP format).
<code>JWTSecret</code>	Bắt buộc	—	Khóa bí mật kiểm tra tính hợp lệ của JWT.
<code>ShortURLBase</code>	Tùy chọn	<code>http://localhost:8080</code>	URL cơ sở ghép với short code trả về client.
<code>IPHashSalt</code>	Tùy chọn	<code>default-salt</code>	Muối băm IP. Cảnh báo : Cần cấu hình lại trên Production để tránh rainbow table attack.
<code>Port</code>	Tùy chọn	<code>8080</code>	Cổng HTTP server.
<code>ServiceName</code>	Hardcode	<code>url-service</code>	Định danh phục vụ ghi log.

Đánh giá: Hệ thống áp dụng nguyên tắc fail-fast khi khởi tạo: mọi lỗi phân tích hoặc thiếu cấu hình bắt buộc sẽ lập tức dừng chương trình.

2.2. Tham Số Kết Nối Các Dịch Vụ Hạ Tầng

Hệ thống thiết lập các kết nối hạ tầng với cơ chế chịu lỗi cụ thể như sau:

Dịch vụ	Tham số kết nối chính	Cơ chế chịu lỗi & Hành vi
PostgreSQL	<code>MaxConns = 10</code> , <code>MinConns = 2</code> , <code>Timeout ping = 10s</code>	Fatal : Dừng dịch vụ lập tức nếu không kết nối được DB (Fail-fast). Tự động dọn dẹp pool khi lỗi.

Dịch vụ	Tham số kết nối chính	Cơ chế chịu lỗi & Hành vi
Redis	Timeout ping = 3s	Non-fatal (Fail-Open) : Redis lỗi không làm sập dịch vụ. Tự động bypass qua cache để truy vấn trực tiếp DB.
RabbitMQ	Max attempts = 10, Exchange = <code>url-shortener</code> (Topic, Durable)	Fatal sau 10 lần thử lại kết nối bằng cơ chế Exponential Backoff (chờ từ 1s, nhân đôi mỗi lần, cap ở 30s).

3. Thiết Kế Cơ Sở Dữ Liệu (Schema Database)

3.1. Bảng `urls` (Quản lý liên kết)

Lưu trữ thông tin liên kết giữa mã rút gọn và URL gốc.

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
<code>id</code>	<code>UUID</code>	PRIMARY KEY DEFAULT <code>gen_random_uuid()</code>	Khóa chính của bản ghi
<code>short_code</code>	<code>VARCHAR(10)</code>	UNIQUE NOT NULL	Mã rút gọn duy nhất (thực tế dài 7 ký tự)
<code>original_url</code>	<code>TEXT</code>	NOT NULL	URL gốc cần rút gọn
<code>user_id</code>	<code>UUID</code>	NOT NULL	Mã người dùng sở hữu URL
<code>user_email</code>	<code>TEXT</code>	NOT NULL DEFAULT ''	Email của người dùng tạo link
<code>created_at</code>	<code>TIMESTAMPTZ</code>	NOT NULL DEFAULT <code>now()</code>	Thời điểm khởi tạo link
<code>expires_at</code>	<code>TIMESTAMPTZ</code>	NULL	Thời điểm hết hạn (NULL là vô hạn)
<code>is_active</code>	<code>BOOLEAN</code>	NOT NULL DEFAULT <code>true</code>	Trạng thái kích hoạt (soft delete)

3.2. Bảng `outbox` (Hàng đợi sự kiện DB)

Lưu trữ các sự kiện tạm thời cần gửi đi các microservices khác qua Message Broker.

Cột	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	PRIMARY KEY DEFAULT gen_random_uuid()	Khóa chính sự kiện
event_type	TEXT	NOT NULL	Loại sự kiện (Ví dụ: url.created, url.deleted)
payload	JSONB	NOT NULL	Nội dung chi tiết của sự kiện dạng JSON
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Thời gian tạo sự kiện
locked_until	TIMESTAMPTZ	NULL	Thời gian khóa bản ghi khi được worker xử lý
published_at	TIMESTAMPTZ	NULL	Thời gian sự kiện được gửi đi thành công

3.3. Chỉ Mục Cơ Sở Dữ Liệu (Indexes)

Hệ thống thiết lập các chỉ mục để tối ưu hóa hiệu năng truy vấn cho hai bảng:

Tên Chỉ Mục	Bảng	Loại chỉ mục	Cột dữ liệu	Mục tiêu tối ưu
idx_urls_short_code	urls	Unique B-Tree	short_code	Đẩy nhanh tốc độ tìm kiếm bản ghi điều hướng theo mã short code.
idx_urls_user_id_created	urls	Composite B-Tree	(user_id, created_at DESC)	Phục vụ truy vấn danh sách liên kết của người dùng có sắp xếp phân trang.
idx_outbox_unpublished	outbox	Partial B-Tree	created_at	Quét nhanh các sự kiện chưa được gửi (published_at IS NULL).
idx_outbox_unpublished_unlocked	outbox	Partial B-Tree	created_at	Quét nhanh các sự kiện chưa gửi

Tên Chi Mục	Bảng	Loại chi mục	Cột dữ liệu	Mục tiêu tối ưu
				và chưa bị khóa (<code>published_at IS NULL AND locked_until IS NULL</code>).

4. Các Hợp Phần Tiện Ích & Helper

4.1. Mã Hóa & Sinh Mã Rút Gọn (Base62 & Code Generator)

- **Thuật toán Base62:** Sử dụng bảng chữ cái `0-9a-zA-Z` (62 ký tự). Hệ thống băm số nguyên lớn (`big.Int`) thu được từ nguồn sinh ngẫu nhiên, thực hiện modulo với (62^7) để giới hạn độ dài chuỗi rút gọn cố định là 7 ký tự (bù thêm ký tự `'0'` ở bên trái nếu thiếu). Khả năng lưu trữ đạt xấp xỉ $(62^7 \approx 3.5)$ nghìn tỷ mã độc nhất.
- **Sinh mã Short Code:** Sinh ngẫu nhiên 8 bytes dữ liệu bằng `crypto/rand` (sử dụng entropy hệ điều hành). Trực tiếp kích hoạt `panic` dừng hệ thống nếu lỗi entropy xảy ra. Xác suất va chạm mã cực kỳ thấp (sau 1 triệu URLs, tỷ lệ trùng mã qua 3 lần thử lại là $(\approx 2.34 \times 10^{-20})$).

4.2. Xác Thực URL & Ánh Xạ Lỗi (Validation & Error Mapping)

- **Quy tắc xác thực:** URL đầu vào bắt buộc phải phân tích cú pháp hợp lệ thông qua `net/url`, scheme phải là `http` hoặc `https` (phòng chống script độc hại), và phải có thông tin host.
- **Ánh xạ lỗi HTTP:** Hệ thống định nghĩa các lỗi logic nội bộ và chuyển đổi sang HTTP Status tương ứng:
 - `ErrInvalidURL` $\rightarrow$ 400 Bad Request
 - `ErrAlreadyExists` $\rightarrow$ 409 Conflict (trùng lặp short code)
 - `ErrNotFound` $\rightarrow$ 404 Not Found
 - `ErrForbidden` $\rightarrow$ 403 Forbidden (sai chủ sở hữu)
 - `ErrExpired` / `ErrDeactivated` $\rightarrow$ 410 Gone (hết hạn hoặc bị xóa)
 - `ErrDatabaseError` $\rightarrow$ 500 Internal Server Error

5. Tầng Lưu Trữ & Hạ Tầng Dữ Liệu

Lớp này quản lý trực tiếp giao tiếp vật lý với các dịch vụ lưu trữ dữ liệu (PostgreSQL, Redis) và Message Broker (RabbitMQ).

5.1. Tầng lưu trữ URLs (URL Store Layer)

Thông qua interface `URLStore` để thực thi CRUD dữ liệu trên PostgreSQL.

- **Insert** : Ghi thông tin URL mới vào database. Thao tác này bắt buộc phải truyền kèm `pgx.Tx` để đảm bảo tính nguyên tử (Atomicity) trong transaction cùng sự kiện outbox.
- **FindByCode** : Đọc thông tin URL thông qua short code ngoài transaction để tối ưu hóa hiệu năng đọc.
- **Deactivate** : Hủy kích hoạt liên kết, cập nhật trạng thái `is_active = false` sau khi so khớp `user_id` sở hữu.
- **Phân trang Cursor**: Truy vấn danh sách URL của user (`FindByUserID`) sử dụng trường UUID `id` làm con trỏ cursor để định vị trang tiếp theo (`id < cursorID`), lấy dư 1 bản ghi để xác định trạng thái còn trang (`hasMore`).

5.2. Tầng lưu trữ sự kiện (Outbox Store Layer)

Quản lý trạng thái và ghi nhận các domain events cần đồng bộ sang RabbitMQ.

- **Insert Event**: Hỗ trợ ghi sự kiện tạo/hủy link trong transaction chính của URL, hoặc ghi sự kiện phân tích click chuột (`url.clicked`) trực tiếp không cần transaction để giảm thiểu tải khóa trên database.
- **Truy vấn Lock-Free với CTE**: Quá trình quét bản ghi chưa gửi sử dụng câu lệnh SQL kết hợp Common Table Expression (CTE) và từ khóa `FOR UPDATE SKIP LOCKED` . Worker sẽ tự động khóa 50 sự kiện trong 30 giây để xử lý, các replica chạy song song sẽ tự động bỏ qua các bản ghi này mà không xảy ra tranh chấp khóa hay trùng lặp tác vụ.

5.3. Tầng Cache đệm (Redis Cache Layer)

Lớp đệm Redis cache triển khai interface `Cache` để tăng tốc truy xuất.

- **Thiết kế**: Lưu trữ đầy đủ thông tin URL gốc, trạng thái hoạt động và thời hạn dùng để phục vụ điều hướng tức thời mà không cần chạm tới database.
- **Resilience (Fail-Open)**: Khi đọc cache đệm, hệ thống thiết lập giới hạn timeout là 50ms. Nếu Redis phản hồi chậm hoặc lỗi kết nối, hệ thống lập tức bỏ qua cache và truy cập trực tiếp PostgreSQL DB.

- **Invalidation:** Lệnh xóa cache được kích hoạt ngay khi người dùng yêu cầu hủy hoạt động của liên kết (Deactivate).

5.4. Tầng Publisher (RabbitMQ Publisher)

Đảm nhiệm việc phát các sự kiện outbox lên RabbitMQ.

- **Đảm bảo độ bền:** Thiết lập thuộc tính `amqp.Persistent` để RabbitMQ ghi dữ liệu xuống đĩa cứng, bảo đảm không thất thoát tin nhắn khi Broker gặp sự cố sập nguồn.
 - **Thread-safe:** Áp dụng khóa đồng bộ `sync.Mutex` trên kênh truyền tải để ngăn chặn các luồng goroutine ghi đè lên nhau gây lỗi channel protocol.
-

6. Tầng Xử Lý Logic & Bootstrapping

Tầng này chịu trách nhiệm điều hành, điều phối nghiệp vụ và khởi tạo vòng đời hoạt động của ứng dụng.

6.1. Tầng dịch vụ nghiệp vụ (Service Layer)

Kết nối các tầng lưu trữ, cache và publisher để thực hiện logic cốt lõi.

- **Luồng Tạo Link (ShortenURL):** Thực hiện validate URL, chuẩn hóa thời gian sống (mặc định 24h, tối đa 1 năm). Thực hiện giao dịch DB ghi nhận thông tin URL và sự kiện Outbox. Nếu xảy ra va chạm khóa trùng lặp (23505), transaction sẽ rollback, chương trình tạm dừng tăng dần và thử lại tối đa 3 lần trước khi báo lỗi 409 Conflict. Cuối cùng, thực hiện ghi đệm cache bất đồng bộ.
- **Luồng Điều hướng (RedirectToURL):** Kiểm tra L1 cache (Redis) trước với timeout 50ms. Nếu hụt cache (MISS), truy vấn PostgreSQL DB rồi cập nhật lại cache. Đồng thời đẩy bất đồng bộ sự kiện click (`url.clicked`) vào bảng outbox để thống kê.
- **Liệt kê & Vô hiệu hóa:** Thực hiện phân trang cursor để trả về danh sách link của người dùng. Thực hiện cập nhật trạng thái hủy kích hoạt trong DB và xóa cache đệm tương ứng.

6.2. Bộ điều phối sự kiện (Outbox Coordinator)

Đóng vai trò background worker quét và gửi tin nhắn.

- **Thông số:** Định kỳ 2 giây quét DB một lần, lấy tối đa 50 sự kiện và phân phối cho 3 worker goroutines xử lý song song.

- **At-Least-Once Delivery:** Workers gửi sự kiện qua RabbitMQ và chỉ đánh dấu `published_at` trong DB sau khi nhận phản hồi thành công từ Broker. Nếu việc cập nhật DB thất bại, sự kiện vẫn nằm lại trong bảng outbox và tự động được gửi lại khi hết hạn khóa 30 giây (duplicate message có thể xảy ra, consumer cần deduplicate).

6.3. HTTP Handler Layer

Tiếp nhận và định tuyến các request từ bên ngoài.

- **Xác thực:** Trích xuất claims `user_id` và `email` từ token JWT đã được xác thực bởi Gateway.
- **Điều hướng HTTP:** Trả về mã trạng thái `308 Permanent Redirect` để trình duyệt lưu cache đích đến, đồng thời kích hoạt luồng phụ ghi nhận thống kê click chuột bất đồng bộ chạy trên context độc lập (`context.Background()`).
- **Nặc danh:** Cấp mã định danh UUID ngẫu nhiên cho user không đăng nhập để cho phép họ rút gọn link mà không cần tài khoản.

6.4. Điểm khởi chạy & Shutdown (Entry Point)

Quản lý khởi động và dọn dẹp hệ thống trong hàm main.

- **Bootstrapping:** Áp dụng cơ chế Manual Dependency Injection để khởi tạo các store, cache, publisher và controller. Tự động đọc và chạy cơ sở dữ liệu từ tệp nhúng tại thời điểm compile (`// go:embed`).
- **Đóng hệ thống an toàn (Graceful Shutdown):** Khi nhận tín hiệu SIGINT/SIGTERM từ hệ điều hành, root context sẽ bị hủy để dừng hoạt động quét của Outbox Coordinator. Tiếp tục dành 10 giây timeout để HTTP Server hoàn tất xử lý các kết nối hiện tại trước khi đóng hoàn toàn và giải phóng kết nối PostgreSQL, Redis, RabbitMQ.

7. Bản Đồ Luồng Hoạt Động & Hiệu Năng

7.1. Quy trình rút gọn link (Shorten URL Flow)

```
sequenceDiagram
    autonumber
    actor Client
    participant H as HTTP Handler
    participant S as URL Service
    participant DB as PostgreSQL
    participant R as Redis
```

```

Client→H: POST /shorten {url, expires_in_hours}
H→S: ValidateURL(url)
alt URL không hợp lệ
    S-->H: Trả về lỗi (400 Bad Request)
    H-->Client: Phản hồi lỗi
else URL hợp lệ
    H→S: ShortenURL()
    loop Thử lại tối đa 3 lần
        S→S: Sinh Short Code ngẫu nhiên (crypto/rand + base62)
        S→DB: Mở Transaction
        S→DB: Chèn bản ghi URL (INSERT url)
        S→DB: Chèn bản ghi Outbox (INSERT outbox)
        alt Lỗi trùng khóa (mã code đã tồn tại)
            DB-->S: Lỗi trùng lặp (Unique Violation 23505)
            S→DB: Rollback Transaction
            Note over S: Ngủ (lần thử * 50ms) và thực hiện lại
        else Thành công
            S→DB: Commit Transaction
            Note over S: Thoát vòng lặp
        end
    end
    alt Vượt quá số lần thử lại
        S-->H: Trả về lỗi (409 Conflict)
        H-->Client: Phản hồi lỗi
    else Thành công
        S→R: Ghi nhận cache đệm (fire-and-forget)
        S-->H: Kết quả rút gọn
        H-->Client: HTTP 201 Created (Kèm short URL)
    end
end
end

```

7.2. Quy trình điều hướng (Redirect Flow)

```

sequenceDiagram
    autonumber
    actor Client
    participant H as HTTP Handler
    participant S as URL Service
    participant R as Redis
    participant DB as PostgreSQL

    Client→>H: GET /{code}
    H→>S: RedirectToURL(code, remoteAddr)

    rect rgb(240, 240, 240)
        Note over S, R: Đọc từ L1 Cache (giới hạn timeout 50ms)
        S→>R: Lệnh Get(code)
        alt Cache HIT
            R-->>S: Trả về dữ liệu CachedURL
            Note over S: Kiểm tra trạng thái is_active & expires_at
        else Cache MISS hoặc Lỗi kết nối Redis
            R-->>S: Trả về rỗng (fallback sang DB)
            S→>DB: Tìm kiếm theo mã (FindByCode)
            DB-->>S: Trả về URLRecord
            S→>R: Ghi nhận cache đệm (fire-and-forget)
        end
    end
end
end

```

```

alt Liên kết bị hủy kích hoạt hoặc hết hạn
  S-->>H: Trả về lỗi (410 Gone)
  H-->>Client: HTTP 410 Gone
else Liên kết hoạt động bình thường
  S-->>H: Thông tin RedirectInfo
  H-->>H: Gọi writeAnalyticsEvent() (chạy ngầm goroutine)
  Note over H: Lưu trữ sự kiện click chuột vào bảng Outbox
  H-->>Client: HTTP 308 Permanent Redirect (Về URL gốc)
end

```

7.3. Quy trình đồng bộ sự kiện (Outbox Processing Flow)

```

flowchart TD
    subgraph Poller [Vòng lặp quét dữ liệu - Định kỳ 2s]
        Start([Khởi động chu kỳ]) --> Fetch[Thực hiện FetchUnpublished]
        Fetch --> CTE["Truy vấn CTE: SELECT FOR UPDATE SKIP LOCKED (Giới hạn 50)"]
        CTE --> Lock["Cập nhật locked_until = hiện tại + 30s"]
        Lock --> Ret["Trả về tập bản ghi"]
        Ret --> SendCh["Đẩy bản ghi vào Jobs Channel"]
    end

    subgraph Workers [Worker Pool - Duy trì 3 Workers]
        JobCh[("Jobs Channel (Hàng đợi đệm 50)")] --> W1[Worker 1]
        JobCh --> W2[Worker 2]
        JobCh --> W3[Worker 3]

        W1 --> Pub1[Publish sự kiện sang RabbitMQ]
        W2 --> Pub2[Publish sự kiện sang RabbitMQ]
        W3 --> Pub3[Publish sự kiện sang RabbitMQ]

        Pub1 --> Mark1[Đánh dấu published_at trong DB]
        Pub2 --> Mark2[Đánh dấu published_at trong DB]
        Pub3 --> Mark3[Đánh dấu published_at trong DB]
    end

    SendCh ~~~ JobCh

```

7.4. Đánh giá hiệu năng và độ trễ (Latency Estimation)

Nghịệp vụ	Chi tiết thao tác	Ước lượng độ trễ
Rút gọn link	Xác thực URL + Sinh mã ngẫu nhiên + Ghi Transaction DB + Thiết lập Cache Redis ngầm	~5 - 25 mili-giây
Điều hướng (Cache HIT)	Truy xuất và kiểm tra thông tin trên Redis cache L1	~1 - 5 mili-giây
Điều hướng (Cache MISS)	Đọc Redis lỗi/hụt → Truy vấn PostgreSQL DB → Thiết lập cache Redis ngầm	~6 - 25 mili-giây

Nghệp vụ	Chi tiết thao tác	Ước lượng độ trễ
Đồng bộ Outbox	Quét DB tìm sự kiện + Gửi qua RabbitMQ + Đánh dấu đã gửi thành công	~150 - 750 mili-giây / Lô 50 sự kiện

8. Phân Tích Phi Chức Năng (Bảo Mật, Tin Cậy, Mở Rộng)

8.1. Đánh giá bảo mật & Lỗ hổng

- **Điểm mạnh:** Thuật toán sinh mã ngẫu nhiên bảo mật; Phân quyền kiểm tra sở hữu ở mức câu lệnh SQL (`WHERE short_code = $1 AND user_id = $2`); Băm một chiều địa chỉ IP kết hợp salt; Parameterized queries ngăn chặn SQL Injection.
- **Lỗ hổng hiện tại:** Sử dụng salt IP mặc định trong mã nguồn (`default-salt`); Không có giới hạn tần suất yêu cầu (Rate Limiting) ở API tạo link; Cổng tạo link nặc danh không yêu cầu xác thực dễ bị spam liên kết xấu.

8.2. Đảm bảo độ tin cậy và Khả năng mở rộng

- **Độ tin cậy cao:** Transactional Outbox bảo đảm không mất sự kiện ngay cả khi RabbitMQ bị sập nguồn; Cơ chế fail-open tự động bypass cache Redis; Giao tiếp tin nhắn ở dạng persistent.
- **Khả năng mở rộng (Scaling):** URL Service stateless hoàn toàn nên có thể dễ dàng tăng replica sau Load Balancer. Cơ chế `FOR UPDATE SKIP LOCKED` cho phép nhiều replica chạy Outbox Coordinator song song quét cùng một DB mà không lo nghẽn hay trùng lặp tác vụ.
- **Điểm nghẽn tiềm ẩn (Bottlenecks):** Khóa mutex trên kênh RabbitMQ publisher giới hạn tốc độ đẩy tin của nhiều worker. Pool kết nối DB mặc định ở mức thấp (10) có thể bị quá tải khi có lượng traffic lớn.

8.3. Mô hình hóa Transactional Outbox Pattern

Mô hình thiết kế này đảm bảo tính nhất quán tuyệt đối giữa dữ liệu nghiệp vụ của liên kết và việc phát các sự kiện liên quan ra hệ thống microservices.

```

flowchart TD
    subgraph DB_Tx [Database Transaction]
        InsertURL["Ghi thông tin URL mới (INSERT urls)"]
        InsertOutbox["Ghi sự kiện nghiệp vụ tạo mới (INSERT outbox)"]
        InsertURL --- InsertOutbox
    end

```

```
Commit([COMMIT Transaction])
Coordinator["OutboxCoordinator (Quét định kỳ 2s)"]
RabbitMQ["Đẩy sự kiện lên RabbitMQ"]

DB_Tx → Commit
Commit → Coordinator
Coordinator → RabbitMQ
```

- **Mục đích của băm IP (hashIP):** Địa chỉ IP client được lọc bỏ cổng, sau đó băm một chiều SHA-256 kết hợp muối cấu hình nhằm mục tiêu đếm chính xác lượt click độc nhất (Unique clicks) mà không làm rò rỉ dữ liệu cá nhân hay vi phạm luật bảo mật thông tin (GDPR).

9. Tích Hợp Shared Packages

Microservice tái sử dụng mã nguồn dùng chung từ các thư viện nội bộ monorepo:

- **Xác thực (Shared Auth Package):** Cung cấp middleware trích xuất, xác thực Bearer Token JWT từ tiêu đề Authorization và đưa thông tin giải mã vào context phục vụ kiểm tra phân quyền.
- **Định nghĩa sự kiện (Shared Events Package):** Đồng bộ hóa cấu trúc sự kiện chung gồm `BaseEvent` (chứa UUID của sự kiện để chống trùng lặp dữ liệu tại consumer), `URLCreatedEvent` (tạo link), `URLDeletedEvent` (hủy kích hoạt link) và `URLClickedEvent` (click chuột).
- **Log cấu trúc (Shared Logger Package):** Cấu hình ghi log JSON sử dụng thư viện chuẩn `slog` của Go, tự động trích xuất mã theo dõi `Correlation ID` từ request context giúp đồng bộ hóa log phân tán xuyên suốt các microservices.

10. Kết Luận

URL Service được thiết kế theo đúng quy chuẩn cloud-native và microservices tiên tiến:

- **Ưu điểm:** Khả năng chịu lỗi cao (Fail-open Redis cache), cơ chế xử lý outbox hiệu năng cao với `SKIP LOCKED`, bảo mật dữ liệu IP cá nhân tốt và cơ chế kiểm soát transaction tin cậy.
- **Khuyến nghị:** Cần bổ sung ngay tăng Rate Limiting (ví dụ: thuật toán Token Bucket) để bảo vệ ứng dụng khỏi spam requests; Chuyển đổi mã định danh cơ sở dữ liệu sang định dạng UUIDv7 để sắp xếp vật lý chỉ mục PostgreSQL hiệu quả hơn, nâng cao throughput ghi dữ liệu.

Báo cáo kiến trúc được tối ưu hóa và định dạng lại dựa trên bộ quy tắc chuẩn của dự án.

Báo Cáo Thiết Kế Kiến Trúc: Event-Driven Architecture, Analytics & Notification Services

Mục Lục

- Tổng Quan về Event-Driven Architecture (EDA)
- Định Nghĩa Event Types
- Cấu Hình RabbitMQ (Exchange, Queue, Binding, Routing)
- Sơ Đồ Luồng Sự Kiện (Event Flow Diagrams)
- Phân Tích Kiến Trúc Analytics Service
- Phân Tích Kiến Trúc Notification Service
- Phân Tích Delivery Guarantees
- Phân Tích Reconnection Handling
- Thiết Kế Stats API Endpoints
- Bảng Tổng Kết Hệ Thống
- Kết Luận & Đề Xuất Cải Tiến

1. Tổng Quan về Event-Driven Architecture (EDA)

1.1. Giới thiệu

Hệ thống URL Shortener Microservices sử dụng **Event-Driven Architecture (EDA)** làm mô hình giao tiếp bất đồng bộ (asynchronous communication) chính giữa các dịch vụ thay vì gọi HTTP API trực tiếp. Mô hình này giúp tăng tính loose coupling (liên kết lỏng lẻo), cho phép mở rộng độc lập (independent scaling) và tối ưu khả năng chịu lỗi (fault tolerance).

1.2. Các thành phần chính trong EDA

Thành phần	Mô tả
Event Producers	

Thành phần	Mô tả
	Các service phát sinh sự kiện (URL Service — tạo/xóa URL, URL Service (qua Outbox Coordinator) — click URL)
Message Broker	RabbitMQ — exchange topic <code>url-shortener</code> , chịu trách nhiệm nhận, định tuyến, và lưu trữ message
Event Consumers	Các service nhận và xử lý sự kiện (Analytics Service, Notification Service)
Event Schema	Định nghĩa cấu trúc dữ liệu của từng loại sự kiện (<code>shared/events/events.go</code>)
Dead Letter / Retry	(Chưa triển khai — sẽ phân tích ở phần Reconnection Handling)

1.3. Nguyên lý hoạt động

1. **Producer** gửi sự kiện lên RabbitMQ Exchange đi kèm một Routing Key tương ứng.
2. **RabbitMQ Topic Exchange** đối chiếu Routing Key với các cấu hình Binding để chuyển hướng tin nhắn về đúng hàng đợi (queue) đã đăng ký.
3. **Consumer** lấy tin nhắn từ hàng đợi, thực hiện logic nghiệp vụ và phản hồi xác nhận xử lý (**ACK/NACK**).
4. Tin nhắn xử lý thành công sẽ bị xóa khỏi hàng đợi. Tin nhắn thất bại sẽ được đưa lại hàng đợi (requeue) tùy theo chính sách lỗi.

1.4. Lợi ích kiến trúc

- **Phân rã dịch vụ (Decoupling):** URL Service hoàn toàn độc lập với Analytics hay Notification Services.
- **Tự do mở rộng (Scalability):** Có thể mở rộng số lượng các instance của Analytics Service theo mô hình Competing Consumers mà không ảnh hưởng tới các dịch vụ khác.
- **Khả năng chịu lỗi (Fault Tolerance):** Nếu Notification Service tạm dừng hoạt động, tin nhắn vẫn tồn tại an toàn trên đĩa cứng của RabbitMQ và được xử lý lại khi dịch vụ trực tuyến.
- **Bất đồng bộ hóa (Asynchrony):** Xử lý lưu trữ thống kê click không làm nghẽn luồng chuyển hướng của khách hàng tại Gateway.

2. Định Nghĩa Event Types

2.1. Định Nghĩa Schema Dùng Chung

Các cấu trúc sự kiện hoạt động như một giao ước chung giữa các dịch vụ, đảm bảo tính nhất quán của dữ liệu truyền tải qua Message Broker.

2.2. Danh Sách Event Types

Tên Sự Kiện	Routing Key	Mô Tả
EventTypeURLCreated	url.created	Kích hoạt khi một mã URL rút gọn mới được tạo
EventTypeURLClicked	url.clicked	Kích hoạt khi người dùng click vào mã URL rút gọn
EventTypeURLDeleted	url.deleted	Kích hoạt khi một mã URL rút gọn bị xóa khỏi hệ thống
EventTypeMilestoneReached	milestone.reached	Kích hoạt khi lượt click của URL đạt các mốc quy định

2.3. BaseEvent (Envelope Sự Kiện Chung)

Tất cả các loại sự kiện đều chứa tập thuộc tính cơ bản để phục vụ việc định tuyến và tracking:

Field	Kiểu	Ý nghĩa
event_type	string	Loại sự kiện, giúp consumer phân biệt mà không cần parse toàn bộ payload
occurred_at	time.Time	Thời điểm sự kiện được tạo (UTC)
correlation_id	string	ID xuyên suốt (tracing), giúp theo dõi một request từ đầu đến cuối qua nhiều service
event_id	string	UUID v4 duy nhất cho mỗi event — dùng để deduplication

2.4. URLCreatedEvent

- **Producer:** URL Service
- **Consumer:** Notification Service

- **Cấu trúc trường mở rộng:**

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
short_code	String	Mã rút gọn của URL
original_url	String	Đường dẫn URL gốc
user_id	String	ID người dùng tạo mã
user_email	String	Email người dùng (phi chuẩn hóa nhằm gửi email trực tiếp không cần truy vấn User Svc)
expires_at	DateTime (Control)	Thời điểm hết hạn của URL (nếu có)

2.5. URLClickedEvent

- **Producer:** URL Service (qua Outbox Coordinator)
- **Consumer:** Analytics Service
- **Cấu trúc trường mở rộng:**

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
short_code	String	Mã rút gọn được kích hoạt
user_id	String	ID của chủ sở hữu URL
user_email	String	Email của chủ sở hữu URL
ip_hash	String	IP người dùng đã băm muối (Salted Hash) nhằm đảm bảo bảo mật và tuân thủ GDPR
user_agent	String	Thông tin trình duyệt và hệ điều hành của client
referrer	String	Nguồn liên kết giới thiệu (HTTP Referrer)
clicked_at	DateTime	Thời điểm người dùng thực hiện click

2.6. URLDeletedEvent

- **Producer:** URL Service
- **Consumer:** Notification Service
- **Cấu trúc trường mở rộng:**

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
short_code	String	Mã rút gọn bị xóa
user_id	String	ID người dùng sở hữu URL
user_email	String	Email người dùng sở hữu URL

2.7. MilestoneReachedEvent

- **Producer:** Analytics Service
- **Consumer:** Notification Service
- **Cấu trúc trường mở rộng:**

Thuộc Tính	Kiểu Dữ Liệu	Ý Nghĩa
short_code	String	Mã rút gọn đạt mốc click
user_id	String	ID chủ sở hữu URL
user_email	String	Email chủ sở hữu URL
milestone	Integer	Giá trị mốc đạt được (Ví dụ: 10, 100, 1000)
total_clicks	Long	Tổng số click hiện tại ghi nhận trong DB

2.8. Bảng Tổng Hợp Event Types Định Tuyến

Event Type	Producer	Consumer	Queue	Routing Key
url.created	URL Service	Notification Service	notifications.events	url.created
url.clicked	URL Service (qua Outbox Coordinator)	Analytics Service	analytics.clicks	url.clicked
url.deleted	URL Service	Notification Service	notifications.events	url.deleted
milestone.reached	Analytics Service	Notification Service	notifications.events	milestone.reached

3. Cấu Hình RabbitMQ (Exchange, Queue, Binding, Routing)

3.1. Exchange Topic

Hệ thống sử dụng một Exchange duy nhất kiểu **Topic** với tên `url-shortener`. * **Durable:** `true` (Tự phục hồi cấu hình Exchange sau khi RabbitMQ restart). * **Auto-Delete:** `false` (Không tự động xóa Exchange khi không còn queue liên kết). * **Lý do chọn Topic:** Cho phép linh hoạt định tuyến thông qua các mẫu ký tự đại diện (*, #). Hỗ trợ thêm mới consumer trong tương lai mà không cần cấu trúc lại code của publisher.

3.2. Analytics Queue

- **Tên hàng đợi:** `analytics.clicks`
- **Durable:** `true` (Tin nhắn chưa xử lý được lưu trữ bền vững trên đĩa).
- **Binding Key:** `url.clicked` (Chỉ lắng nghe duy nhất sự kiện click URL).

3.3. Notification Queue

- **Tên hàng đợi:** `notifications.events`
- **Durable:** `true`
- **Binding Keys:** Bind đồng thời với 3 Routing Keys:
 - `url.created`
 - `url.deleted`
 - `milestone.reached`

3.4. Sơ Đồ Routing

```
graph TD
    classDef broker fill:#f9f,stroke:#333,stroke-width:2px;
    classDef service fill:#bbf,stroke:#333,stroke-width:2px;

    Exchange["url-shortener<br>(Topic Exchange)"] ::: broker
    QueueAnalytics["analytics.clicks<br>(Durable Queue)"] ::: broker
    QueueNotification["notifications.events<br>(Durable Queue)"] ::: broker
    AnalyticsSvc["Analytics Service<br>(ClickConsumer)"] ::: service
    NotificationSvc["Notification Service<br>(NotificationConsumer)"] ::: service

    Exchange -->|"routing: url.clicked"| QueueAnalytics
    Exchange -->|"routing: url.created"| QueueNotification
    Exchange -->|"routing: url.deleted"| QueueNotification
    Exchange -->|"routing: milestone.reached"| QueueNotification
```

```
QueueAnalytics → AnalyticsSvc
QueueNotification → NotificationSvc
```

3.5. AMQP Prefetch (QoS)

Cấu hình **QoS Prefetch Count = 1** được thiết lập trên cả hai consumers. * **Ý nghĩa:** Mỗi instance consumer chỉ được nhận tối đa 1 tin nhắn tại một thời điểm từ RabbitMQ. Nó bắt buộc phải phản hồi ACK/NACK trước khi nhận tin nhắn tiếp theo. * **Mục đích:** Đảm bảo xử lý tuần tự, cân bằng tải động (Fair Dispatch) giữa các consumers đang chạy song song, giảm nguy cơ quá tải bộ nhớ đệm của ứng dụng.

3.6. Message Persistence

- **DeliveryMode = 2 (Persistent):** Toàn bộ tin nhắn được xuất bản từ các Publisher (URL Service (qua Outbox Coordinator), Analytics Service) đều được cấu hình lưu đĩa. Kết hợp với hàng đợi Durable, hệ thống đảm bảo an toàn dữ liệu ngay cả khi cụm RabbitMQ gặp sự cố sập nguồn đột ngột.

4. Sơ Đồ Luồng Sự Kiện (Event Flow Diagrams)

4.1. Sơ Đồ Luồng Sự Kiện Click (Luồng Chính)

Luồng xử lý bất đồng bộ từ lúc Gateway phát sinh click cho đến khi Analytics Service cập nhật dữ liệu, kiểm tra cột mốc và gửi tín hiệu mốc click:

```
sequenceDiagram
    autonumber
    participant "URL Service" as URLSvc
    participant Outbox
    participant RabbitMQ
    participant Analytics as Analytics Service
    participant PostgreSQL

    URLSvc->>Outbox: Insert URLLClickedEvent (Transactional Outbox)
    Outbox->>RabbitMQ: Publish URLLClickedEvent (routing: "url.clicked")
    RabbitMQ->>Analytics: Phân phối tới queue "analytics.clicks"
    Note over Analytics: Giải mã JSON & Kiểm tra tính hợp lệ dữ liệu
    Note over Analytics: Khởi tạo DATABASE TRANSACTION
    Analytics->>PostgreSQL: SELECT EXISTS trong processed_events (event_id)
    PostgreSQL-->>Analytics: exists = false (Sự kiện chưa được xử lý)
    Analytics->>PostgreSQL: INSERT INTO processed_events (event_id)
    Analytics->>PostgreSQL: INSERT INTO clicks (short_code, clicked_at, ip_hash, ...)
    Note over Analytics: Logic kiểm tra mốc Click (CheckAndPublish)
```



```

Analytics->>PostgreSQL: SELECT COUNT(*) FROM clicks WHERE short_code
PostgreSQL-->>Analytics: Trả về total_clicks (Ví dụ: 100)
Analytics->>PostgreSQL: SELECT EXISTS trong milestones WHERE milestone=100
PostgreSQL-->>Analytics: exists = false (Mốc này chưa từng đạt được)
Analytics->>PostgreSQL: INSERT INTO milestones (short_code, milestone=100)
Analytics->>RabbitMQ: Publish MilestoneReachedEvent (routing:
"milestone.reached")
RabbitMQ-->>Analytics: Nhận sự kiện & Định tuyến tới notifications.events
queue
Analytics->>PostgreSQL: COMMIT TRANSACTION
Analytics->>RabbitMQ: Phản hồi ACK (Xóa tin nhắn khỏi queue
analytics.clicks)

```

4.2. Sơ Đồ Luồng Sự Kiện Notification

Luồng lưu trữ thông báo và giả lập gửi email cho khách hàng khi nhận các sự kiện hệ thống:

```

sequenceDiagram
    autonumber
    participant Source as URL Service / Analytics
    participant RabbitMQ
    participant Notification as Notification Service
    participant PostgreSQL

    Source->>RabbitMQ: Publish Event (url.created / url.deleted /
milestone.reached)
    RabbitMQ->>Notification: Phân phối tới queue "notifications.events"
    Note over Notification: Giải mã JSON & Đọc Event Type từ Routing Key
    Note over Notification: Khởi tạo DATABASE TRANSACTION
    Notification->>PostgreSQL: INSERT INTO notifications (status='pending')
    PostgreSQL-->>Notification: Trả về notification_id và created_at
    Note over Notification: Ghi nhận Log giả lập gửi email (Mock Email Sent)
    Notification->>PostgreSQL: UPDATE notifications SET status='sent',
sent_at=now() WHERE id
    Notification->>PostgreSQL: COMMIT TRANSACTION
    Notification->>RabbitMQ: Phản hồi ACK (Xóa tin nhắn khỏi queue
notifications.events)

```

5. Phân Tích Kiến Trúc Analytics Service

5.1. Tổng Quan

Dịch vụ Analytics chịu trách nhiệm xử lý toàn bộ dữ liệu thống kê liên quan đến hành vi tương tác mã URL của người dùng, phân tích xu hướng truy cập theo nguồn giới thiệu (referrer), cung cấp trực thời gian click (timeline) và kích hoạt các cột mốc thành tựu của URL.

5.2. Cấu Trúc Thành Phần

Dịch vụ được phân chia thành các cấu phần chức năng rõ rệt:

Thành phần	Vai trò kiến trúc
Khởi tạo hệ thống	Cấu hình tham số môi trường, thiết lập kết nối cơ sở dữ liệu và RabbitMQ, chạy các script Migration tự động
Bộ nhận tin nhắn (Consumer)	Triển khai consumer xử lý các sự kiện click từ hàng đợi RabbitMQ
Logic Cột mốc (Milestone)	Kiểm tra các ngưỡng giới hạn (10, 100, 1000) và xuất bản sự kiện đạt mốc
Lớp dữ liệu (Store Layer)	Thực hiện các truy vấn đọc/ghi tối ưu hóa xuống PostgreSQL sử dụng Connection Pool
Bộ xử lý API (Handler)	Expose các REST API để phục vụ ứng dụng frontend truy vấn thống kê dữ liệu

5.3. Quy Trình Khởi Tạo và Cấu Hình Thời Gian Chờ (Timeout)

Ứng dụng áp dụng quy trình khởi chạy tuần tự và cơ chế tự động thử lại kết nối (Exponential Backoff Retry) để đảm bảo tính sẵn sàng trước khi lắng nghe sự kiện:

Tham số cấu hình	Giá trị mặc định	Vai trò
databaseStartupTimeout	60 giây	Thời gian tối đa chờ khởi động và kết nối thành công tới Database
rabbitMQStartupTimeout	120 giây	Thời gian tối đa để thử kết nối tới RabbitMQ Cluster
shutdownTimeout	10 giây	Thời gian để kết thúc các tiến trình đang chạy khi nhận tín hiệu SIGTERM/SIGINT
rabbitMQAttempts	10 lần	Số lần thử kết nối lại tối đa trước khi ứng dụng tự ngắt (crash/exit)

5.4. ClickConsumer — Bộ Nhận và Xử Lý Sự Kiện Click

5.4.1. Tham số Đăng Ký Lắng Nghe (Consume)

Khi đăng ký lắng nghe hàng đợi `analytics.clicks`, các tham số cấu hình kết nối được thiết lập chi tiết:

Parameter	Giá trị	Ý nghĩa
queue	"analytics.clicks"	Tên hàng đợi cần tiêu thụ
consumer	"" (empty)	RabbitMQ tự sinh tag định danh duy nhất cho Consumer này
autoAck	false	Manual ACK — consumer chủ động gửi ACK/NACK sau khi xử lý xong
exclusive	false	Cho phép nhiều consumer cùng truy cập song song (competing consumers)
noLocal	false	Nhận cả message được tạo ra từ chính connection này
noWait	false	Đợi broker xác nhận đăng ký thành công
args	nil	Không sử dụng cấu hình bổ sung

- **Cơ chế giao dịch (Transaction Scope):** Tất cả các tiến trình ghi nhận click gồm: kiểm tra sự kiện trùng lặp (Dedup Check), chèn bản ghi đã xử lý (Processed Event), chèn thông tin lượt click (Clicks Table) và cập nhật cột mốc (Milestone Update) được gói gọn trong một Database Transaction duy nhất. Thiết kế này loại bỏ khả năng bất nhất dữ liệu nếu xảy ra lỗi giữa chừng.
- **Xử lý Poison Message:** Nếu sự kiện không thể giải mã JSON hoặc thiếu các thông tin bắt buộc (Event ID, Short Code), hệ thống sẽ chủ động phản hồi **ACK** để hủy bỏ tin nhắn đó, ngăn chặn vòng lặp xử lý lỗi vô hạn làm nghẽn hàng đợi (Poison Message Loop).
- **Phục hồi sự cố (Panic Recovery):** Sử dụng cơ chế phục hồi động (defer recover). Nếu xảy ra lỗi nghiêm trọng runtime (Panic), consumer sẽ tự phục hồi và phản hồi ACK để hủy bỏ tin nhắn, chấp nhận mất mát dữ liệu nhỏ để bảo toàn tính hoạt động liên tục của dịch vụ.

5.5. MilestoneChecker — Logic Kiểm Tra và Ghi Nhận Mốc Click

- **Mốc kiểm tra:** [10, 100, 1000]
- **Kiến trúc tránh trùng lặp cột mốc (Idempotent Milestone):** 1. Đếm số click thực tế của short code trong Transaction hiện tại. 2. Sử dụng câu lệnh `INSERT ... ON CONFLICT (short_code, milestone) DO NOTHING` để chống ghi đè hoặc tạo trùng lặp bản ghi cột mốc do tranh chấp luồng (Race Condition). 3. Chỉ khi bản ghi cột mốc được chèn thành công mới phát đi sự kiện `MilestoneReachedEvent`.
- **Thiết kế xử lý lỗi gửi sự kiện:** Nếu việc xuất bản sự kiện `MilestoneReachedEvent` lên RabbitMQ bị lỗi, dịch vụ chỉ ghi nhận cảnh báo (Warn Log) và vẫn hoàn thành giao dịch (Commit Transaction). Điều này ưu tiên tính toàn vẹn của dữ liệu click thay vì rollback toàn bộ giao dịch chỉ vì broker gặp sự cố tạm thời.

5.6. AnalyticsPublisher — Bộ Phát Sự Kiện Mốc Click

- Gửi sự kiện lên Exchange `url-shortener` với Routing Key `milestone.reached`.
- Cấu hình tin nhắn bền vững (**Persistent Mode**).
- Sử dụng Context có giới hạn thời gian (**Timeout 3 giây**) để giải phóng tài nguyên hệ thống nếu Broker không phản hồi kịp thời.

5.7. Lớp Lưu Trữ Dữ Liệu (Store Layer)

Lớp dữ liệu sử dụng thư viện kết nối Pool tối ưu hóa, thực hiện các truy vấn thông qua các chỉ mục (Indexes) đã được tính toán kỹ lưỡng:

Tên Truy Vấn / Thao Tác	Nội Dung SQL Tóm Tắt	Mục Đích	Index Được Sử Dụng
Insert Click	<code>INSERT INTO clicks (short_code, clicked_at, ip_hash, user_agent, referer) VALUES (...)</code>	Ghi nhận một lượt click mới	-
Count Clicks	<code>SELECT COUNT(*) FROM clicks WHERE short_code = \$1</code>	Đếm tổng số click của một mã URL	<code>idx_clicks_short_code_time</code>
Count Clicks Since	<code>SELECT COUNT(*) FROM clicks WHERE short_code = \$1 AND clicked_at ≥ \$2</code>	Đếm số click trong một khoảng thời gian (24h, 7d)	<code>idx_clicks_short_code_time</code>
Top Referers	<code>SELECT referer, COUNT(*) FROM clicks WHERE short_code = \$1 GROUP BY referer ORDER BY cnt DESC LIMIT \$2</code>	Liệt kê nguồn giới thiệu hàng đầu	<code>idx_clicks_referer</code> (Partial Index)
Timeline Buckets	<code>SELECT date_trunc(\$1, clicked_at) AS period, COUNT(*) FROM clicks WHERE short_code = \$2 GROUP BY period ORDER BY period ASC</code>	Phân nhóm thống kê theo giờ/ngày	<code>idx_clicks_short_code_time</code>
Check Milestone	<code>SELECT EXISTS(SELECT 1 FROM milestones WHERE</code>		<code>idx_milestones_code_milestone</code>

Tên Truy Vấn / Thao Tác	Nội Dung SQL Tóm Tắt	Mục Đích	Index Được Sử Dụng
	<code>short_code = \$1 AND milestone = \$2)</code>	Kiểm tra mốc click đã đạt được chưa	
Insert Milestone	<code>INSERT INTO milestones (short_code, milestone) VALUES (...) ON CONFLICT DO NOTHING</code>	Ghi nhận mốc click mới (idempotent)	<code>idx_milestones_code_milestone</code>
Check Processed Event	<code>SELECT EXISTS(SELECT 1 FROM processed_events WHERE event_id = \$1)</code>	Kiểm tra sự kiện đã xử lý chưa (dedup)	PK của <code>processed_events</code>
Insert Processed Event	<code>INSERT INTO processed_events (event_id) VALUES (...) ON CONFLICT DO NOTHING</code>	Ghi nhận sự kiện đã xử lý	PK của <code>processed_events</code>

5.8. Cấu Trúc Bảng Cơ Sở Dữ Liệu

5.8.1 Bảng `clicks`

Bảng lưu trữ vết truy cập chi tiết của người dùng.

Cột	Kiểu	Ràng buộc	Mô tả
<code>id</code>	UUID	PRIMARY KEY, mặc định <code>gen_random_uuid()</code>	ID tự sinh cho mỗi click
<code>short_code</code>	TEXT	NOT NULL	Mã rút gọn của URL
<code>clicked_at</code>	TIMESTAMPTZ	NOT NULL	Thời điểm xảy ra click
<code>ip_hash</code>	TEXT	NOT NULL	Giá trị IP đã được băm bảo mật
<code>user_agent</code>	TEXT	NOT NULL, mặc định <code>''</code>	Trình duyệt người dùng

Cột	Kiểu	Ràng buộc	Mô tả
referrer	TEXT	NULL	Nguồn giới thiệu

• **Khóa chỉ mục:**

- `idx_clicks_short_code_time` trên (`short_code`, `clicked_at` DESC) : Composite index phục vụ đếm và phân trang thời gian.
- `idx_clicks_referer` trên (`short_code`, `referrer`) WHERE `referrer` IS NOT NULL : Partial index giúp tăng tốc truy vấn thống kê referer và tiết kiệm không gian lưu trữ do bỏ qua các click không có referer.

5.8.2 Bảng `milestones`

Lưu trữ lịch sử đặt mốc của các mã URL rút gọn.

Cột	Kiểu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY, mặc định <code>gen_random_uuid()</code>	ID duy nhất của cột mốc đặt được
short_code	TEXT	NOT NULL	Mã rút gọn đặt mốc
milestone	INT	NOT NULL	Ngưỡng mốc click đặt (10, 100, 1000)
triggered_at	TIMESTAMPTZ	NOT NULL, mặc định <code>now()</code>	Thời gian đặt mốc

- **Ràng buộc nghiệp vụ:** Ràng buộc độc nhất `UNIQUE(short_code, milestone)` đảm bảo mỗi mốc click chỉ được kích hoạt một lần duy nhất cho mỗi URL.

5.8.3 Bảng `processed_events`

Bảng phục vụ kiểm tra trùng lặp sự kiện (Deduplication Log).

Cột	Kiểu	Ràng buộc	Mô tả
event_id	TEXT	PRIMARY KEY	UUID v4 độc nhất của sự kiện click
processed_at	TIMESTAMPTZ	NOT NULL, mặc định <code>now()</code>	Thời điểm xử lý xong sự kiện

- *Lưu ý kiến trúc:* Hiện tại bảng này chưa cấu hình thời gian sống (TTL), về lâu dài cần thiết lập cơ chế dọn dẹp định kỳ để tránh phình to dung lượng dữ liệu.

6. Phân Tích Kiến Trúc Notification Service

6.1. Tổng Quan

Dịch vụ thông báo đảm nhận vai trò cầu nối thông tin tới người dùng. Dịch vụ thu nhận các biến động hệ thống (Tạo URL mới, xóa URL cũ, đặt cột mốc tương tác), lưu trữ lịch sử thông báo và thực thi việc gửi thông tin phản hồi (trong phiên bản hiện tại là ghi log giả lập email).

6.2. Cấu Trúc Thành Phần

Notification Service được tổ chức gọn nhẹ với các cấu phần: * **Khởi tạo hệ thống:** Thiết lập môi trường và cấu hình tự động migration cơ sở dữ liệu. * **Consumer:** Lắng nghe 3 sự kiện chính cấu hình trong Queue. * **Store Layer:** Thực thi các tác vụ ghi thông báo và đọc danh sách thông báo phân trang. * **Handler:** Cung cấp HTTP API bảo vệ bằng JWT để người dùng lấy danh sách thông báo cá nhân.

6.3. Quy Trình Khởi Tạo Hệ Thống

Quy trình tương đồng với Analytics Service, sử dụng cơ chế Exponential Backoff để tăng tính tự phục hồi khi khởi chạy.

6.4. NotificationConsumer — Bộ Nhận Sự Kiện Thông Báo

- **Sử dụng Routing Key phân nhánh:** Consumer đọc giá trị `RoutingKey` từ siêu dữ liệu của Broker để ánh xạ kiểu sự kiện thay vì giải mã toàn bộ JSON payload trước. Điều này tăng hiệu năng xử lý đáng kể.

6.4.1 Bảng so sánh cơ chế Consumer

Aspect	ClickConsumer	NotificationConsumer
ACK strategy	Manual (<code>autoAck = false</code>)	Manual (<code>autoAck = false</code>)
Validation	Parse + validate các trường bắt buộc	Parse + validate các trường bắt buộc
Invalid message	Phản hồi ACK và loại bỏ	Phản hồi ACK và loại bỏ
Error during processing	Phản hồi NACK và requeue	Phản hồi NACK và requeue
Transaction		

Aspect	ClickConsumer	NotificationConsumer
	Mở/đóng trực tiếp ở Consumer xử lý	Thực thi bên trong hàm store repository
Deduplication	Có (bảng <code>processed_events</code>)	Không có

6.5. Lớp Lưu Trữ Thông Báo

6.5.1 Tiến trình giao dịch tạo thông báo:

1. Tạo bản ghi thông báo mới ở trạng thái chờ gửi (`status = 'pending'`).
2. Thực thi giả lập gửi email thông qua hệ thống Logs (`Mock Email Sent`).
3. Cập nhật trạng thái thông báo thành hoàn thành (`status = 'sent'`) kèm thời gian gửi (`sent_at = now()`).
4. Commit toàn bộ giao dịch để đảm bảo ghi nhận đồng bộ.

6.5.2 Phân trang danh sách thông báo (Cursor-based Pagination):

Để phục vụ danh sách thông báo hiệu năng cao cho người dùng, dịch vụ sử dụng phân trang dạng Keyset (Cursor-based) thay vì Offset truyền thống. * **Cursor được chọn:** Cặp giá trị kết hợp (`created_at, id`). * **So sánh cơ chế phân trang:**

Aspect	Offset-based	Keyset (Cursor-based)
Performance	Suy giảm nghiêm trọng khi offset lớn (tải trang sâu)	Tốc độ truy xuất ổn định $O(\log n)$
Stability	Thao tác chèn mới trong lúc đọc dễ làm lệch trang, lặp bản ghi	Ổn định tuyệt đối, không bị trùng/lặp bản ghi
Implementation	Đơn giản, dễ viết	Phức tạp hơn (sử dụng so sánh giá trị cặp trường)
Use case	Phù hợp cho hiển thị UI đơn giản, ít cập nhật	Phù hợp cho hệ thống Real-time Feed, API phân trang vô hạn

6.6. Cấu Trúc Bảng Cố Định

Bảng `notifications`

Cột	Kiểu	Ràng buộc	Mô tả
<code>id</code>	UUID		ID duy nhất của thông báo

Cột	Kiểu	Ràng buộc	Mô tả
		PRIMARY KEY, mặc định <code>gen_random_uuid()</code>	
<code>user_id</code>	UUID	NOT NULL	ID người dùng nhận thông báo
<code>event_type</code>	TEXT	NOT NULL	Loại sự kiện phát sinh
<code>payload</code>	JSONB	NOT NULL	Toàn bộ thông tin gốc JSON của sự kiện
<code>status</code>	TEXT	NOT NULL, mặc định <code>'sent'</code>	Trạng thái gửi thông báo (<code>pending</code> / <code>sent</code>)
<code>created_at</code>	TIMESTAMPTZ	NOT NULL, mặc định <code>now()</code>	Thời điểm tạo thông báo
<code>sent_at</code>	TIMESTAMPTZ	NULL	Thời điểm gửi email thực tế

- **Chỉ mục:** `idx_notifications_user_created` trên `(user_id, created_at DESC)` hỗ trợ tối ưu hóa tối đa truy vấn phân trang của người dùng.

6.7. Bộ Xử Lý Endpoint API (Handler)

- Endpoint chính: `GET /notifications?limit=20&after=<cursor_id>`
- Giới hạn kích thước trang (Limit Cap): Tối đa `100` bản ghi trên một truy vấn để tránh tấn công từ chối dịch vụ (DoS) qua API.
- Xác thực: Bắt buộc đính kèm JWT Bearer Token trong header HTTP.

6.8. Mô-đun Xác Thực Hệ Thống

- Middleware giải mã JWT và lưu trữ thông tin User Claims (`sub` làm `user_id` , `email`) vào HTTP Request Context.
 - Các API handlers phía sau trích xuất an toàn `user_id` để thực thi phân quyền dữ liệu, đảm bảo người dùng không thể truy cập trái phép thông báo của người khác.
-

7. Phân Tích Delivery Guarantees

7.1. Các Mức Đảm Bảo Delivery

Mức	Mô tả	Ví dụ
At-most-once	Message được gửi tối đa một lần. Có thể mất message nhưng không có duplicate.	Fire-and-forget, auto-ACK trước khi xử lý
At-least-once	Message được gửi ít nhất một lần. Không mất message nhưng có thể có duplicate.	Manual ACK sau khi xử lý thành công
Exactly-once	Message được gửi đúng một lần. Không mất, không duplicate.	At-least-once kết hợp deduplication

7.2. Analytics Service Delivery Guarantee

Đạt mức **Exactly-once** nhờ thiết kế phối hợp:

flowchart TD
Producer([Producer]) --> RabbitMQ[RabbitMQ]
RabbitMQ --> Consumer[Consumer (autoAck=false)]
Consumer --> Check{Kiểm tra Event ID đã tồn tại trong DB?}
Check --> Đã tồn tại AckDiscard[Phản hồi ACK & Bỏ qua xử lý]
Check --> Chưa tồn tại Process[Bắt đầu Xử lý sự kiện]
Process --> DBTx[Database Transaction]
DBTx --> Ghi dữ liệu thành công Commit[Commit Transaction]
Commit --> Ack[Phản hồi ACK gửi Broker]
DBTx --> Lỗi ghi dữ liệu Rollback[Rollback Transaction]
Rollback --> Nack[Phản hồi NACK & Đưa lại Queue]

7.3. Notification Service Delivery Guarantee

Đạt mức **At-least-once**. * Không có cơ chế Deduplication riêng biệt tại tầng nhận thông báo. * Nếu consumer xảy ra sự cố đột ngột sau khi đã ghi nhận cơ sở dữ liệu thành công nhưng chưa kịp gửi phản hồi ACK về cho RabbitMQ, tin nhắn sẽ được Broker đưa lại hàng đợi và phân phối lại. Điều này dẫn tới khả năng lặp thông báo (Duplicate Notification) đối với người dùng cuối.

7.4. So Sánh Đảm Bảo Delivery

Service	autoAck	Manual ACK	Dedup	Guarantee	Rủi ro
Analytics	false				

Service	autoAck	Manual ACK	Dedup	Guarantee	Rủi ro
		ACK sau commit	processed_events table	Exactly-once	Mất message nếu panic (ACK trong recoverDeliveryPanic)
Notification	false	ACK sau insert	Không có	At-least-once	Duplicate notification

7.5. Xử Lý Poison Message

Hệ thống áp dụng các phản ứng khác nhau đối với lỗi phát sinh trong luồng xử lý sự kiện:

Loại lỗi	Analytics	Notification
Invalid JSON	ACK (discard)	ACK (discard)
Missing required field	ACK (discard)	ACK (discard)
Unsupported routing key	N/A	ACK (discard)
Panic	ACK (discard)	ACK (discard)
DB error	NACK requeue	NACK requeue

8. Phân Tích Reconnection Handling

8.1. Kết Nối Khi Khởi Chạy

Hệ thống sử dụng Exponential Backoff để kết nối với RabbitMQ khi khởi chạy: * Khoảng thời gian giữa cách khởi đầu: 1 giây. * Cơ chế nhân đôi giữa cách sau mỗi lần thử sai:

1s → 2s → 4s → 8s → 16s → 30s... (Giới hạn tối đa 30 giây). * Số lần thử tối đa: 10 lần.

8.2. Mất Kết Nối Khi Đang Chạy — Vấn Đề Thiết Kế

- **Hiện trạng sự cố:** Khi kết nối tới RabbitMQ Cluster bị đứt lúc dịch vụ đang hoạt động ổn định (Runtime), kênh phân phối tin nhắn (deliveries) sẽ bị đóng phát đi tín hiệu ok = false.

- **Hậu quả:** Cả hai dịch vụ Analytics và Notification Consumers đều chỉ thiết lập biến trạng thái sức khỏe `healthy = false` và chuyển sang trạng thái treo chờ Context bị hủy bỏ. Dịch vụ **hoàn toàn ngừng việc tiêu thụ tin nhắn vĩnh viễn** cho tới khi quản trị viên thực hiện khởi động lại container thủ công. Dữ liệu trong hàng đợi RabbitMQ sẽ tích lũy gây phình to bộ nhớ đệm của Broker.

8.3. Giải Pháp Tự Động Kết Nối Lại Đề Xuất

Để khắc phục lỗi thiết kế trên, cần nâng cấp quy trình xử lý mất kết nối tự động (Self-Healing Connection):

1. **Đăng ký sự kiện đóng kết nối:** Sử dụng hàm lắng nghe thông báo đóng kênh (`conn.NotifyClose`) và đóng kết nối (`channel.NotifyClose`).
2. **Khởi động vòng lặp kết nối lại (Reconnection Loop):** Khi nhận tín hiệu đứt kết nối, kích hoạt tiến trình thiết lập lại kết nối mới sau một khoảng thời gian chờ tăng dần.
3. **Đăng ký lại Consumer:** Sau khi kết nối thành công, tiến hành khai báo lại Exchange, các Queue, thiết lập lại QoS và đăng ký lắng nghe kênh phân phối (`Consume`) mới.

8.4. So Sánh Với URL Service

URL Service đóng vai trò là Publisher (Người phát tin nhắn). Khi mất kết nối RabbitMQ, nó sẽ trả về lỗi HTTP 500 trực tiếp cho các yêu cầu tạo/xóa URL. Mặc dù làm giảm trải nghiệm người dùng, lỗi này vẫn có thể khắc phục được bằng cách người dùng thực hiện gửi lại request (Client Retry). Với tầng Consumer, việc mất kết nối âm thầm mà không tự hồi phục có mức độ nghiêm trọng cao hơn nhiều do làm gián đoạn hoàn toàn luồng xử lý bất đồng bộ phía sau.

9. Thiết Kế Stats API Endpoints

9.1. Endpoint Kiểm Tra Sức Khỏe

- **Đường dẫn:** `GET /health`
- **Cơ chế:** Trả về mã JSON tình đã được mã hóa trước để tăng tối đa hiệu năng phản hồi. Tuy nhiên, endpoint này chưa tích hợp kiểm tra kết nối thực tế tới Database hay RabbitMQ (chỉ là Liveness Check chứ chưa đạt chuẩn Readiness Check).

9.2. Endpoint Đo Lường Chỉ Số Metrics

- **Đường dẫn:** `GET /metrics`

- **Mô tả:** Expose dữ liệu đo lường theo chuẩn Prometheus phục vụ hệ thống giám sát hiệu năng trực quan.

9.3. Endpoint Thống Kê Tổng Quan URL

- **Đường dẫn:** `GET /stats/{code}`
- **Cơ chế xử lý song song (Concurrency Model):** Để tối ưu hóa tốc độ phản hồi API, dịch vụ sử dụng mô hình đồng thời `errgroup` để thực thi song song 4 câu lệnh truy vấn dữ liệu độc lập: 1. Đếm tổng lượt click. 2. Đếm số lượt click trong vòng 24 giờ qua. 3. Đếm số lượt click trong vòng 7 ngày qua. 4. Lấy danh sách 5 nguồn giới thiệu hàng đầu (Top Referers).
- Nếu bất kỳ truy vấn nào gặp lỗi, toàn bộ tiến trình sẽ bị hủy bỏ ngay lập tức để tiết kiệm tài nguyên hệ thống (Fail-Fast).

9.4. Endpoint Lược Sử Theo Trục Thời Gian

- **Đường dẫn:** `GET /stats/{code}/timeline?interval=day`
- **Tham số phân nhóm (Interval):** Bắt buộc phải là `"day"` hoặc `"hour"`.
- **Cơ chế tối ưu hóa DB:** Sử dụng hàm `date_trunc` của PostgreSQL thực thi trực tiếp tại tầng cơ sở dữ liệu để phân nhóm thời gian theo múi giờ UTC, tránh việc tải lượng lớn dữ liệu thô về bộ nhớ của ứng dụng để phân tích.
- **Hạn chế:** API chưa hỗ trợ tham số lọc khoảng thời gian (`from` và `to`), dẫn đến nguy cơ trả về payload dung lượng quá lớn nếu URL có lịch sử hoạt động lâu năm.

9.5. Endpoint Xem Danh Sách Thông Báo

- **Đường dẫn:** `GET /notifications`
- **Xác thực:** JWT Bearer Token.
- **Cơ chế phân trang:** Keyset Pagination (Cursor) sử dụng UUID.

10. Bảng Tổng Kết Hệ Thống

10.1. So Sánh Hai Service

Aspect	Analytics Service	Notification Service
Primary function	Xử lý click events, thống kê	Xử lý notification events, mock email

Aspect	Analytics Service	Notification Service
Queue	<code>analytics.clicks</code>	<code>notifications.events</code>
Routing keys	1: <code>url.clicked</code>	3: <code>url.created</code> , <code>url.deleted</code> , <code>milestone.reached</code>
autoAck	<code>false</code>	<code>false</code>
Dedup	Có (processed_events table)	Không
Delivery guarantee	Exactly-once	At-least-once
API endpoints	<code>/stats/{code}</code> , <code>/stats/{code}/timeline</code>	<code>/notifications</code>
API auth	Không	JWT required
Transaction scope	Consumer (1 transaction)	Store (1 transaction)
Producer	Có (MilestoneReachedEvent)	Không
DB tables	<code>clicks</code> , <code>milestones</code> , <code>processed_events</code>	<code>notifications</code>
DB indexes	2 composite, 1 partial	1 composite
Migration	SQL embedding (<code>go:embed</code>)	SQL embedding (<code>go:embed</code>)

10.2. Cấu Hình Hàng Đợi RabbitMQ

Item	Giá trị
Exchange name	<code>url-shortener</code>
Exchange type	<code>topic</code>
Durable	<code>true</code>
Auto-delete	<code>false</code>
Queue analytics	<code>analytics.clicks</code> (durable)
Queue notification	<code>notifications.events</code> (durable)
Routing keys (analytics)	<code>url.clicked</code>
Routing keys (notification)	<code>url.created</code> , <code>url.deleted</code> , <code>milestone.reached</code>

Item	Giá trị
Prefetch count (analytics)	1
Prefetch count (notification)	1

10.3. Bảng Tóm Tắt Thuộc Tính Events Payload

Event	Producer	Routing Key	Queues	Payload Fields
URLCreatedEvent	URL Service	url.created	notifications.events	ShortCode, OriginalURL, UserID, userEmail, ExpiresAt
URLClickedEvent	URL Service (qua Outbox Coordinator)	url.clicked	analytics.clicks	ShortCode, UserID, userEmail, IPHash, UserAgent, Referer, ClickedAt
URLDeletedEvent	URL Service	url.deleted	notifications.events	ShortCode, UserID, userEmail
MilestoneReachedEvent	Analytics Svc	milestone.reached	notifications.events	ShortCode, UserID, userEmail, MilestoneN, TotalClicks

10.4. Các Bảng Cơ Sở Dữ Liệu Tóm Tắt

Service	Table	PK	Unique Constraints	Indexes
Analytics	clicks	id (UUID)	-	(short_code, clicked_at DESC), (short_code, referer) WHERE referer IS NOT NULL

Service	Table	PK	Unique Constraints	Indexes
Analytics	milestones	id (UUID)	(short_code, milestone)	(short_code, milestone)
Analytics	processed_events	event_id (TEXT)	-	-
Notification	notifications	id (UUID)	-	(user_id, created_at DESC)

10.5. Chiến Lược Xử Lý Lỗi Và Định Tuyến Hàng Đợi

Scenario	Analytics	Notification
Invalid JSON	ACK + log và loại bỏ tin	ACK + log và loại bỏ tin
Missing required field	ACK + log và loại bỏ tin	ACK + log và loại bỏ tin
DB error (transient)	NACK requeue thử lại sau	NACK requeue thử lại sau
DB error (permanent)	NACK requeue (loop vô hạn)	NACK requeue (loop vô hạn)
Panic	ACK + log + recover tự phục hồi	ACK + log + recover tự phục hồi
Duplicate event	ACK bỏ qua (đã ghi nhận DB)	Không áp dụng (ghi nhận lại tin)
Unsupported event type	N/A	ACK + log bỏ qua
Milestone publish fail	WARN + commit giao dịch	N/A

10.6. Cấu Hình Biến Môi Trường (Environment Variables)

Variable	Analytics	Notification	Ý nghĩa kiến trúc
DATABASE_URL	Bắt buộc	Bắt buộc	Chuỗi kết nối tới cơ sở dữ liệu PostgreSQL
RABBITMQ_URL	Bắt buộc	Bắt buộc	Chuỗi kết nối tới Broker RabbitMQ
PORT	Optional (Mặc định 8080)	Optional (Mặc định 8080)	Cổng lắng nghe của HTTP Server
JWT_SECRET	Không dùng	Bắt buộc	Khóa giải mã chữ ký số JWT của người dùng

Variable	Analytics	Notification	Ý nghĩa kiến trúc
IP_HASH_SALT	Tùy chọn	Không dùng	Muối (Salt) dùng để băm IP chống dịch ngược

11. Kết Luận & Đề Xuất Cải Tiến

11.1. Điểm Mạnh Kiến Trúc

1. **Thiết kế Event-Driven chuẩn hóa:** Tách biệt rõ ràng trách nhiệm giữa các dịch vụ thông qua Topic Exchange, Queue Durable và Persistent Message.
2. **Exactly-once đảm bảo tại lõi hệ thống:** Đảm bảo thống kê click chính xác tuyệt đối nhờ sự kết hợp giữa Transaction Atomicity và bảng Deduplication Log.
3. **Phân trang Keyset thông minh:** Tối ưu hóa hiệu năng tải danh sách thông báo cho người dùng cuối.
4. **Xử lý Graceful Shutdown hoàn chỉnh:** Giải phóng tài nguyên an toàn khi có tín hiệu dừng hệ thống.

11.2. Điểm Yếu Cần Khắc Phục

1. **Runtime Reconnection (Nghiêm trọng - P0):** Consumer bị treo vĩnh viễn khi mất kết nối RabbitMQ trong lúc vận hành.
2. **Thiếu hàng đợi Dead Letter Queue (P1):** Các tin nhắn gặp lỗi cơ sở dữ liệu tạm thời bị đưa lại hàng đợi và thử lại lập tức liên tục, dễ gây tràn CPU của dịch vụ.
3. **Trùng lặp thông báo (P1):** Notification Service chưa áp dụng cơ chế lọc trùng lặp sự kiện như Analytics Service.
4. **Thiếu cơ chế dọn dẹp (P2):** Bảng `processed_events` phình to không giới hạn theo thời gian.
5. **Độ trễ khi khôi phục sự kiện mốc (P2):** Lỗi gửi sự kiện milestone không được lưu lại để thử lại tự động ở background worker.

11.3. Lộ Trình Cải Tiến Khuyến Nghị

```

timeline
    title Lộ trình nâng cấp kiến trúc hệ thống
    Giai đoạn 1 (Immediate - P0) : Khắc phục lỗi mất kết nối Runtime : Tích hợp NotifyClose tự phục hồi kết nối
    Giai đoạn 2 (High - P1) : Bảo vệ hàng đợi & Chống trùng lặp : Cấu hình Dead Letter Queue (DLQ) cho RabbitMQ : Thêm Deduplication cho Notification
  
```

Service

Giai đoạn 3 (Medium - P2) : Tối ưu hóa dữ liệu & Giám sát : Thêm TTL cleanup job cho bảng processed_events : Nâng cấp API Timeline hỗ trợ lọc khoảng thời gian : Tích hợp kiểm tra Readiness Check thực tế cho Database

Giai đoạn 4 (Low - P3) : Tái cấu trúc mã nguồn : Chuyển các module dùng chung (db, rabbitmq, auth) thành Shared Library

Phụ Lục: Sơ Đồ Luồng Sự Kiện Tổng Thể (Mermaid)

Sơ đồ tổng quan mô tả dòng chảy của các loại sự kiện chạy xuyên suốt qua hệ thống URL Shortener:

flowchart TD

```
classDef broker fill:#f9f,stroke:#333,stroke-width:2px;
classDef service fill:#bbf,stroke:#333,stroke-width:2px;
```

```
subgraph Producers [Nguồn Phát Sự Kiện]
    URLSvc["URL Service"] ::: service
    URLSvcOutbox["URL Service (Outbox)"] ::: service
    AnalyticsSvcPub["Analytics Service (Publisher)"] ::: service
end
```

```
Exchange["url-shortener exchange<br>(Topic Exchange)"] ::: broker
```

```
subgraph Queues [Hàng Đợi RabbitMQ]
    QueueClicks["analytics.clicks queue<br>(Durable)"] ::: broker
    QueueEvents["notifications.events queue<br>(Durable)"] ::: broker
end
```

```
subgraph Consumers [Nguồn Tiêu Thụ Sự Kiện]
    AnalyticsSvcCons["Analytics Service (Consumer)"] ::: service
    NotificationSvcCons["Notification Service (Consumer)"] ::: service
end
```

```
URLSvc -->|"url.created<br>url.deleted"| Exchange
URLSvcOutbox -->|"url.clicked"| Exchange
AnalyticsSvcPub -->|"milestone.reached"| Exchange
```

```
Exchange -->|"routing: url.clicked"| QueueClicks
Exchange -->|"routing: url.created, url.deleted, milestone.reached"| QueueEvents
```

```
QueueClicks --> AnalyticsSvcCons
QueueEvents --> NotificationSvcCons
```

```
%% Ép buộc sắp xếp chồng đứng để tối ưu hóa hiển thị
Producers ~~~ Exchange ~~~ Queues ~~~ Consumers
```

Bảo Mật & Chịu Lỗi (Security & Resilience)

Mục Lục

1. Xác Thực Người Dùng (User Authentication)
 2. JWT Token Management
 3. Mật Khẩu & Bảo Mật
 4. Circuit Breaker (Gateway)
 5. Rate Limiter (Gateway)
 6. Shared Auth Package
-

1. Xác Thực Người Dùng (User Authentication)

What

`user-service` chịu trách nhiệm đăng ký (register) và đăng nhập (login), exposed qua Gateway tại `/api/auth/register` và `/api/auth/login`.

How

- **Password:** bcrypt cost factor 12 (~200ms/hash)
- **Session:** JWT HS256, 24h TTL
- **Lưu trữ:** PostgreSQL `users` table với `password_hash` là bcrypt string

Problem (Cũ) — Timing Attack

Khi email không tồn tại trong DB, handler trả về lỗi ngay lập tức mà không chạy bcrypt verify. Kẻ tấn công đo được: - Email **tồn tại**: response ~200ms (bcrypt verify mất ~200ms) - Email **không tồn tại**: response ~1ms (không chạy bcrypt)

→ Dễ dàng brute-force danh sách email hợp lệ.

Solution — Dummy bcrypt Hash

```
const dummyBcryptHash = "$2a$12$MB4lTvA5UVWJU8GPtVFSne/kMHaXBSz45DWvIL/4AS9NLnz7tavNm"
```

Khi `FindByEmail` trả về `nil`, handler vẫn gọi `hasher.Verify(password, dummyBcryptHash)`:

- Response time đồng nhất ~200ms cho cả email tồn tại và không tồn tại
- Attacker không thể phân biệt dựa trên timing

Register Flow

Bước	Thành phần	Hành động
1	Client	POST /api/auth/register với JSON {email, password}
2	Gateway	Bỏ qua JWT (route public), forward đến user-service
3	Handler	Validate Content-Type, body size (1MB max), email format, password ≥ 8 chars
4	PasswordHasher	<code>bcrypt.GenerateFromPassword(password, cost=12)</code>
5	UserRepository	INSERT INTO users (email, password_hash)
6	—	Nếu <code>ErrDuplicateEmail</code> → 409 Conflict
7	Handler	Trả về 201 Created {user_id, email}

Login Flow

Bước	Thành phần	Hành động
1	Client	POST /api/auth/login với JSON {email, password}
2	Gateway	Bỏ qua JWT, forward đến user-service
3	Handler	Validate request body
4	UserRepository	<code>FindByEmail</code> — truy vấn PostgreSQL
5	—	Nếu <code>user == nil</code> → <code>hasher.Verify(password, dummyBcryptHash)</code> → 401
6	—	Nếu <code>user tồn tại</code> → <code>hasher.Verify(password, user.PasswordHash)</code>
7	—	Nếu <code>password sai</code> → 401

Bước	Thành phần	Hành động
8	TokenIssuer	Sinh JWT HS256 với claims <code>sub</code> , <code>email</code> , <code>iss</code> , <code>iat</code> , <code>exp</code>
9	Handler	Trả về 200 OK <code>{token, expires_at}</code>

2. JWT Token Management

What / How

`jwtTokenIssuer` trong `user-service` sử dụng `golang-jwt/v5` để sinh và verify token:

Thuộc tính	Giá trị
Thuật toán	HS256 (HMAC-SHA256)
Signing key	Symmetric secret (shared giữa <code>user-service</code> và <code>gateway</code>)
TTL	24 giờ
Claims	<code>sub</code> (<code>user_id</code> UUID), <code>email</code> , <code>iss</code> (<code>"url-shortener"</code>), <code>iat</code> , <code>exp</code>

Vấn Đề Hiện Tại

Vấn đề	Mô tả	Mức độ
Symmetric key shared	<code>user-service</code> sinh token, <code>gateway</code> verify — cả 2 dùng chung 1 secret. Nếu 1 service bị lộ, attacker tự do forge token.	Cao
Không có refresh token	Token hết hạn sau 24h, client phải login lại. UX kém cho mobile/long-lived sessions.	Trung bình
No revocation	Không thể thu hồi token trước hạn. Nếu token bị rò rỉ, attacker dùng được đến khi hết hạn.	Cao
Stateless verification	<code>Gateway</code> verify token mà không cần gọi <code>user-service</code> (nhanh) nhưng không check blacklist.	—

Future

- **Refresh token:** Token ngắn hạn (15p) + refresh token dài hạn (7 ngày) lưu trong DB
- **Redis blacklist:** Danh sách token bị thu hồi với TTL = token expiry

3. Mật Khẩu & Bảo Mật

bcrypt Cost Factor

```
func NewPasswordHasher(cost int) PasswordHasher {  
    if cost < bcrypt.MinCost { cost = bcrypt.MinCost }  
    if cost > bcrypt.MaxCost { cost = bcrypt.MaxCost }  
    return &bcryptHasher{cost: cost}  
}
```

Cost	~Thời gian	Bảo mật
10 (default)	~100ms	Cơ bản
12	~200ms	Hiện tại — cân bằng speed/security
14 (max)	~800ms	Quá chậm cho hệ thống production

Giới hạn [MinCost, MaxCost] ngăn config lỗi đặt cost ngoài range.

hashIP (Analytics & URL Service)

SHA-256 + salt để băm IP client trước khi lưu:

- **Mục đích:** Đếm unique clicks mà không lưu PII (Personal Identifiable Information)
- **GDPR compliance:** IP gốc không bao giờ được lưu vào DB
- **Salt:** Cấu hình qua env, tránh rainbow table

SQL Injection Protection

Toàn bộ truy vấn sử dụng **parameterized queries** (pgx — PostgreSQL driver):

```
-- users table migration  
CREATE TABLE IF NOT EXISTS users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    email TEXT UNIQUE NOT NULL,  
    password_hash TEXT NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Mọi INSERT, SELECT, UPDATE đều dùng \$1, \$2, ... — không concatenate chuỗi user input vào SQL.

4. Circuit Breaker (Gateway)

What

Tự triển khai 3-state machine tại Gateway để bảo vệ upstream service khỏi cascading failure.

Problem

Upstream service (vd: url-service) gặp sự cố → Gateway tiếp tục forward request → Chờ timeout
→ Tài nguyên cạn kiệt → Các service khác cũng bị ảnh hưởng (cascading failure).

How — 3-State Machine

State	Hành vi	Điều kiện chuyển
CLOSED	Requests pass through, đếm failure trong <code>failureWindow</code>	—
OPEN	Reject ngay lập tức với <code>ErrCircuitOpen</code> (503)	failures $\geq$ maxFailures (5) trong 10s
HALF_OPEN	Cho phép probe request kiểm tra	OPEN timeout 30s expired + request mới tới

State Diagram

```
stateDiagram-v2
    [*] → CLOSED

    CLOSED → OPEN : 5 failures trong 10s

    OPEN → HALF_OPEN : Sau 30s, request đầu tiên vào

    HALF_OPEN → CLOSED : Probe success
    HALF_OPEN → OPEN : Probe fail (reset timer)
```

Chi Tiết Implementation

Khía cạnh	Mô tả
State struct	<code>CircuitBreaker</code> với <code>sync.Mutex</code> , <code>State</code> , <code>failures</code> , <code>lastFailureTime</code> , <code>maxFailures</code> (5), <code>openTimeout</code> (30s), <code>failureWindow</code> (10s)
Thread safety	<code>sync.Mutex</code> — lock khi đọc/ghi state và counters
Half-open probe	Biến <code>halfOpenProbe</code> bool: khi đang <code>HALF_OPEN</code> , chỉ 1 request được phép pass, các request khác bị reject
Context cancellation	Kiểm tra <code>ctx.Done()</code> trước khi gọi upstream — tránh goroutine leak
State change callback	<code>onStateChange(from, to State)</code> gọi sau khi unlock mutex, dùng cho Prometheus metrics

Metrics (Prometheus)

Metric	Type	Labels
<code>circuit_breaker_state</code>	Gauge	<code>service="url-service"</code> — 0=CLOSED, 1=OPEN, 2=HALF_OPEN
<code>circuit_breaker_trips_total</code>	Counter	<code>service="url-service"</code> — số lần OPEN
<code>circuit_breaker_requests_rejected_total</code>	Counter	<code>service="url-service"</code> — số request bị reject

Hạn Chế

- **Không sliding window thực sự:** `failureWindow` reset toàn bộ counter khi hết window, không phải sliding
 - **Không exponential backoff:** OPEN timeout cố định 30s, không tăng dần theo số lần trip
 - **Chỉ bảo vệ url-service:** Các upstream khác (user-service, notification-service) chưa được cấu hình CB
-

5. Rate Limiter (Gateway)

What

Fixed Window Counter algorithm trên Redis, giới hạn request theo IP.

Problem

Một user gửi request với tần suất cao → Quá tải upstream → Tăng latency, giảm throughput cho user khác.

How — Fixed Window Counter trên Redis

```
// Lua-equivalent logic:
count := redis.INCR("rl:" + key)
if count == 1 { redis.EXPIRE(key, windowSecs) }
if count > limit { REJECT }
```

Endpoint	Limit	Window	Ý nghĩa
shorten	10	60s	Tối đa 10 request mỗi 60 giây
redirect	300	60s	Tối đa 300 request mỗi 60 giây
Redis timeout	100ms	Timeout per Redis call	

Fail-Open

Khi Redis lỗi hoặc timeout, RateLimiter **allow request** và ghi WARN log:

```
count, err := rl.client.Incr(ctx, fullKey).Result()
if err != nil {
    return true, 0, err // fail-open: allow request
}
```

Quyết định thiết kế: **Availability** > **Consistency** — ưu tiên dịch vụ hoạt động hơn là chặn thiếu chính xác.

Xác Định Client IP

```
func clientIP(r *http.Request) string {
    // 1. X-Forwarded-For header (first IP)
```

```
// 2. X-Real-IP header
// 3. RemoteAddr (fallback)
}
```

Hỗ trợ reverse proxy chain (Nginx → Gateway).

Hạn Chế

- **IP-based:** Không phân biệt user trong cùng IP (NAT, shared IP). User hợp lệ có thể bị ảnh hưởng bởi user khác.
- **Không per-endpoint chi tiết:** Chỉ có 2 key (`shorten` , `redirect`). Các endpoints auth (`/register` , `/login`) không có rate limit riêng.
- **Fixed Window:** Dễ bị burst ở boundary (vd: 100 req ở giây cuối + 100 req ở giây đầu của window kế).

6. Shared Auth Package

Claims & VerifyToken

`shared/auth/auth.go` định nghĩa cấu trúc JWT claims chung cho toàn hệ thống:

Field	JSON	Kiểu	Bắt buộc	Mô tả
Sub	<code>sub</code>	<code>string</code> (UUID)	✓	User ID
Email	<code>email</code>	<code>string</code>	✓	Email user (denormalized)
Iss	<code>iss</code>	<code>string</code>	✓	Luôn <code>"url-shortener"</code>
Iat	<code>iat</code>	<code>int64</code> (Unix)	✓	Issued at
Exp	<code>exp</code>	<code>int64</code> (Unix)	✓	Expiry

`VerifyToken` kiểm tra: 1. Algorithm = HMAC (ngăn attack `alg=none`) 2. Signature hợp lệ 3. Tất cả required claims tồn tại và đúng kiểu 4. `iss` = `"url-shortener"`

JWTMiddleware (Gateway)

`shared/auth/middleware.go` — middleware HTTP pattern:

```
func JWTMiddleware(secret string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
```

```

        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            // 1. Extract Bearer token từ Authorization header
            // 2. VerifyToken (signature + claims validation)
            // 3. context.WithValue → inject claims vào request context
            // 4. next.ServeHTTP
        })
    }
}

```

Luồng JWT Middleware

```

sequenceDiagram
    autonumber
    actor Client
    participant G as API Gateway
    participant JWT as JWT Middleware
    participant Upstream as Upstream Service

    Client->>G: POST /api/shorten (Authorization: Bearer <token>)

    rect rgb(255, 240, 240)
        Note over G,JWT: JWT Middleware
        G->>JWT: Extract Bearer token
        JWT->>JWT: VerifyToken(token, secret)
        alt Token hợp lệ
            JWT->>JWT: Inject Claims vào context
            JWT-->>G: Pass (context với claims)
            G->>Upstream: Forward request (context mang claims)
            Upstream->>Upstream: ClaimsFromContext(ctx) → user_id, email
            Upstream-->>G: 200 OK response
            G-->>Client: 200 OK
        else Token hết hạn / sai signature
            JWT-->>G: 401 Unauthorized
            G-->>Client: {"error": "unauthorized"}
        else Không có Authorization header
            JWT-->>G: 401 Unauthorized
            G-->>Client: {"error": "authorization header required"}
        end
    end
end

```

Dual Verification (Defense in Depth)

1. **Gateway:** JWTMiddleware chặn request không hợp lệ trước khi forward
2. **Upstream service:** Mỗi service gọi `ClaimsFromContext` để verify lại — nếu request bypass Gateway (mạng nội bộ), vẫn được bảo vệ

Tổng Kết

Lĩnh vực	Cơ chế chính	Hạn chế / Future
Xác thực	bcrypt cost 12, dummy hash chống timing attack	—
JWT	HS256, 24h TTL, symmetric key	Không refresh token, không revocation
Password	bcrypt [10-14], parameterized queries	—
Circuit Breaker	3-state, 5 failures/10s, 30s timeout	Chỉ url-service, không exponential backoff
Rate Limiter	Fixed Window Counter Redis, fail-open	IP-based, fixed window
Auth middleware	Dual verify (gateway + service)	—

Cơ Sở Dữ Liệu (Database)

Mục Lục

1. Database-per-Service Pattern
 2. Schema từng Service
 3. Indexes
 4. Migration Strategy
 5. Cursor-based Pagination
 6. Docker Compose DB Configuration
-

1. Database-per-Service Pattern

What

4 PostgreSQL instances riêng biệt — mỗi service một database, 4 container, không chia sẻ.

Why

- **Isolation:** Lỗi/service đi xuống không ảnh hưởng dữ liệu service khác
- **Independent scaling:** Mỗi database scale riêng — URL service cần nhiều connection hơn User service
- **Bounded context ownership:** Mỗi team/service tự do chọn schema, index, migration strategy

Trade-off

- Không cross-service JOIN — aggregate ở application layer, query riêng rẽ
- **Eventual consistency:** Không có FK constraint xuyên database — dữ liệu nhất quán cuối cùng qua outbox pattern

Danh sách Database

Database	Service	Container	Port host
urldb	URL Service	url_db	5432
analyticsdb	Analytics Service	analytics_db	5433
usersdb	User Service	user_db	5434
notificationdb	Notification Service	notification_db	5435

```
flowchart LR
    subgraph S["Services"]
        US[URL Service]
        AS[Analytics Service]
        NS[Notification Service]
        URS[User Service]
    end
    subgraph D["Databases"]
        UDB[(urldb<br/>5432)]
        ADB[(analyticsdb<br/>5433)]
        UUDB[(usersdb<br/>5434)]
        NDB[(notificationdb<br/>5435)]
    end
    US --> UDB
    AS --> ADB
    NS --> NDB
    URS --> UUDB
```

2. Schema từng Service

urldb — URL Service

Bảng urls

Cột	Kiểu	RB	Ghi chú
id	UUID	PK, DEFAULT gen_random_uuid()	
short_code	VARCHAR(10)	UNIQUE NOT NULL	Mã rút gọn 7 ký tự
original_url	TEXT	NOT NULL	URL gốc
user_id	UUID	NOT NULL	Chủ sở hữu
user_email	TEXT	NOT NULL DEFAULT "	Email người tạo

Cột	Kiểu	RB	Ghi chú
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
expires_at	TIMESTAMPTZ	NULL	NULL = vô hạn
is_active	BOOLEAN	NOT NULL DEFAULT true	Soft delete

Bảng outbox — Transactional Outbox

Cột	Kiểu	RB	Ghi chú
id	UUID	PK, DEFAULT gen_random_uuid()	
event_type	TEXT	NOT NULL	Routing key, e.g. url.created
payload	JSONB	NOT NULL	Event body
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
locked_until	TIMESTAMPTZ	NULL	Claim lock cho worker
published_at	TIMESTAMPTZ	NULL	NULL = chưa publish

analyticsdb — Analytics Service

Bảng clicks

Cột	Kiểu	RB	Ghi chú
id	UUID	PK, DEFAULT gen_random_uuid()	
short_code	TEXT	NOT NULL	Logical FK → urls.short_code
clicked_at	TIMESTAMPTZ	NOT NULL	
ip_hash	TEXT	NOT NULL	SHA-256(IP + salt)
user_agent	TEXT	NOT NULL DEFAULT "	
referrer	TEXT	NULL	

Bảng milestones — Theo dõi mốc click

Cột	Kiểu	RB
id	UUID	PK, DEFAULT gen_random_uuid()
short_code	TEXT	NOT NULL
milestone	INT	NOT NULL
triggered_at	TIMESTAMPTZ	NOT NULL DEFAULT now()
UNIQUE	(short_code, milestone)	

Bảng processed_events — Idempotency cho consumer

Cột	Kiểu
event_id	TEXT PK
processed_at	TIMESTAMPTZ NOT NULL DEFAULT now()

notificationsdb — Notification Service

Bảng notifications

Cột	Kiểu	RB	Ghi chú
id	UUID	PK, DEFAULT gen_random_uuid()	
user_id	UUID	NOT NULL	Logical FK → users.id
event_type	TEXT	NOT NULL	
payload	JSONB	NOT NULL	Thông tin milestone, URL
status	TEXT	NOT NULL DEFAULT 'sent'	sent / failed / pending
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
sent_at	TIMESTAMPTZ	NULL	

usersdb — User Service

Bảng users

Cột	Kiểu	RB
id	UUID	PK, DEFAULT gen_random_uuid()

Cột	Kiểu	RB
email	TEXT	UNIQUE NOT NULL
password_hash	TEXT	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()

Khác biệt chính giữa các schema

Đặc điểm	urls	users	notifications	clicks
Soft delete	is_active	—	—	—
Expiry	expires_at	—	—	—
State machine	—	—	status	—
JSONB payload	Chỉ outbox	—	payload	—
Logical FK	user_id	—	user_id	short_code
Dùng gen_random_uuid()	Có	Có	Có	Có

Tất cả FK đều **logical** (không DB constraint) — đúng tinh thần database-per-service.

Nhờ gen_random_uuid() ở mọi PK, insert không phụ thuộc sequence.

3. Indexes

Tên Index	Bảng	Cột	Loại	Mục đích
idx_urls_short_code	urls	short_code	UNIQUE B-Tree	Tra cứu redirect $O(\log n)$
idx_urls_user_id_created	urls	(user_id, created_at DESC)	Composite B-Tree	Danh sách user, cursor pagination
idx_outbox_unpublished	outbox	created_at ASC	Partial (published_at IS NULL)	Quét outbox event chưa gửi
idx_outbox_unpublished_unlocked	outbox			

Tên Index	Bảng	Cột	Loại	Mục đích
		created_at ASC	Partial (published_at IS NULL AND locked_until IS NULL)	Quét outbox không lock
idx_clicks_short_code_time	clicks	(short_code, clicked_at DESC)	Composite B- Tree	Thống kê click theo URL
idx_clicks_referer	clicks	(short_code, referer)	Partial (referer IS NOT NULL)	Thống kê referrer
idx_milestones_code_milestone	milestones	(short_code, milestone)	UNIQUE B- Tree	Chống milestone trùng
idx_notifications_user_created	notifications	(user_id, created_at DESC)	Composite B- Tree	Tra notification của user
idx_users_email	users	email	UNIQUE B- Tree	Login, unique constraint

Điểm đáng chú ý

- **Partial indexes trên outbox:** Chỉ index row `published_at IS NULL` — index nhỏ gọn, outbox poller không bao giờ scan published rows
- **Composite + DESC:** `ORDER BY id DESC` với index `(user_id, created_at DESC)` cho phép index-only scan, không cần sort step
- **Không index JSONB payload:** Dữ liệu event chỉ ghi + đọc toàn bộ — không query nội dung

4. Migration Strategy

Cách hoạt động

```
//go:embed migration.sql
var migrationSQL string

func RunMigrations(ctx context.Context, pool *pgxpool.Pool) error {
    _, err := pool.Exec(ctx, migrationSQL)
    return err
}
```

- **Embedded SQL:** File `.sql` nhúng vào binary tại compile time
- **Startup migration:** Chạy ngay khi service init, trước khi serve request
- **Idempotent:** Mọi câu lệnh đều dùng `IF NOT EXISTS` — chạy lại vô hại

Bảng ưu / nhược điểm

Ưu điểm	Nhược điểm
Zero dependency — không cần tool	Không versioning — không biết schema ở version nào
Idempotent — chạy lại an toàn	Chỉ CREATE, không ALTER/DROP — không mutable
Đơn giản — một file SQL, ai cũng hiểu	Không review migration theo PR
Startup-time — DB ready ngay	Không rollback — muốn revert phải manual
Dễ dev — không cần migrate up/down	Scale kém khi nhiều môi trường

Khi nào phù hợp

- **MVP / dự án nhỏ:** Simplicity > control
 - **Schema ổn định:** URL service schema ít thay đổi sau khi định hình
 - **Giới hạn:** Nếu cần ALTER TABLE / nhiều môi trường → chuyển sang `golang-migrate` hoặc `atlas`
-

5. Cursor-based Pagination

Problem với OFFSET

OFFSET 10000 → scan 10000 rows rồi bỏ qua → $O(n)$

- Performance giảm dần theo số trang
- Dữ liệu thay đổi giữa request → page bị lặp/mất row

Solution: Keyset Pagination (URL Service)

```
-- Lấy limit + 1 row để xác định hasMore
WHERE user_id = $1 AND id < $cursor AND is_active = true
      AND (expires_at IS NULL OR expires_at > NOW())
ORDER BY id DESC
LIMIT $limit + 1
```

UUID id làm cursor: - Sort ổn định, không thay đổi - `WHERE id < $cursor` index seek trực tiếp — $O(\log n)$

```
hasMore := len(results) > limit
if hasMore {
    results = results[:limit] // bỏ row thừa
}
```

Hiệu năng

Phương pháp	Page 1	Page 100	Page 10000
OFFSET	~0.3ms	~5ms	~15ms
Cursor	~0.3ms	~0.3ms	~0.3ms

Cursor-based cho performance **ổn định $O(\log n)$** — không degrade theo số lượng data.

6. Docker Compose DB Configuration

Service	Image	Port map	User	Password	Database	Volume
url_db		5432:5432	urluser	urlpass	urldb	url_db_data

Service	Image	Port map	User	Password	Database	Volume
	postgres:16-alpine					
analytics_db	postgres:16-alpine	5433:5432	analyticsuser	analyticspass	analyticsdb	analytics_db_data
user_db	postgres:16-alpine	5434:5432	useruser	userpass	userdb	user_db_data
notification_db	postgres:16-alpine	5435:5432	notificationuser	notificationpass	notificationdb	notification_db_data

Health Check

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U <user> -d <db>"]
  interval: 5s
  timeout: 5s
  retries: 10
  start_period: 10s
```

- **pg_isready**: Kiểm tra qua Unix socket — không cần password
- **start_period 10s**: Cho PostgreSQL init trước khi health check bắt đầu
- **depends_on condition: service_healthy**: Service chỉ start sau khi DB pass health

depends_on chain

```
flowchart LR
  url_db --> url-service
  analytics_db --> analytics-service
  user_db --> user-service
  notification_db --> notification-service
  url-service --> gateway
  analytics-service --> gateway
  user-service --> gateway
  notification-service --> gateway
  gateway --> nginx
```

Tổng kết

Khía cạnh	Lựa chọn	Lý do
Pattern	Database-per-Service	Isolation, bounded context
Migration	Embedded SQL + IF NOT EXISTS	Zero dependency, idempotent
Pagination	Cursor-based	Performance $O(\log n)$, consistent
Index	Partial + Composite B-Tree	Tối ưu query pattern cụ thể
Deploy	Docker Compose + health check	Dev/CI friendly, reproducible

Hạn chế lớn nhất: Migration không versioning. Khi dự án lớn cần multi-environment, nên chuyển sang golang-migrate / atlas để có migrate up/down, review migration file, và rollback an toàn.

Phân Tích Chi Tiết: Deployment, Docker Compose, Kubernetes, CI/CD và Monitoring

Mục Lục

1. Tổng Quan Kiến Trúc Triển Khai
2. Docker Compose & Dockerfiles — Phát Triển & Build - 2.1. Cấu Hình Chi Tiết Các Container - 2.2. Tài Nguyên, Phụ Thuộc và Môi Trường - 2.3. Quy trình Build Docker (Dockerfiles)
3. Kubernetes Manifests - 3.1. Namespace - 3.2. ConfigMap và Secrets - 3.3. Infrastructure Deployments - 3.4. Application Deployments - 3.5. Services và NodePort - 3.6. Phân Tích Deployment vs StatefulSet
4. Health Checks — Liveness và Readiness - 4.1. ReadinessProbe trong Docker Compose - 4.2. ReadinessProbe trong Kubernetes - 4.3. So Sánh và Đối Chiếu
5. Infrastructure Components - 5.1. PostgreSQL (4 Instances) - 5.2. Redis Cache - 5.3. RabbitMQ Message Broker - 5.4. Nginx Reverse Proxy - 5.5. Adminer
6. CI/CD Pipeline — GitHub Actions - 6.1. Continuous Integration (CI) - 6.2. Continuous Delivery (CD) - 6.3. Build Matrix - 6.4. Docker Layer Caching - 6.5. Multi-Arch Build - 6.6. Deploy to AKS
7. Monitoring Stack - 7.1. Prometheus — Cấu Hình Scrape - 7.2. Grafana — Dashboards - 7.3. Loki — Log Aggregation - 7.4. Promtail — Log Collector - 7.5. Datasource Provisioning
8. Logging Stack - 8.1. Loki Configuration - 8.2. Promtail Configuration - 8.3. Grafana Log Explorer
9. Container Dependency Graph - 9.1. Dependency Tree - 9.2. Startup Order - 9.3. Critical Path Analysis
10. Production Deployment Recommendations
 - 10.1. Ingress Controller
 - 10.2. Horizontal Pod Autoscaler (HPA)
 - 10.3. StatefulSet cho Databases
 - 10.4. Resource Requests và Limits
 - 10.5. PersistentVolumeClaims
 - 10.6. Network Policies
 - 10.7. Pod Disruption Budgets
 - 10.8. Secret Management

1. Tổng Quan Kiến Trúc Triển Khai

Dự án URL Shortener Microservices sử dụng kiến trúc microservices với ngôn ngữ Go cho backend (5 services), Node.js/Vite cho frontend, và cơ sở dữ liệu PostgreSQL phân tán (4 databases riêng biệt). Hệ thống áp dụng hai chiến lược triển khai song song:

Môi Trường	Platform	Mục Đích
Development	Docker Compose	18 containers chạy local, phát triển và debug
Production	Kubernetes (AKS)	Triển khai cloud-native, scale tự động

Kiến trúc triển khai tổng thể được biểu diễn như sơ đồ dưới đây:

```
graph TD
    classDef db fill:#85C1E9,stroke:#333,stroke-width:2px;
    classDef svc fill:#82E0AA,stroke:#333,stroke-width:2px;
    classDef broker fill:#F8C471,stroke:#333,stroke-width:2px;
    classDef monitor fill:#D7BDE2,stroke:#333,stroke-width:2px;

    Client[Internet] --> Nginx[Nginx Reverse Proxy]
    Nginx -->|Port 80: /api, /r, /health| Gateway[API Gateway:8080]
    Nginx -->|Port 80: /| Frontend[Frontend:5173]

    Gateway -->|HTTP| URLs[URL Service:8081]:::svc
    Gateway -->|HTTP| AnalyticsS[Analytics Service:8082]:::svc
    Gateway -->|HTTP| UserS[User Service:8083]:::svc
    Gateway -->|HTTP| NotificationS[Notification Service:8084]:::svc

    URLs --> URL_DB[(url_db)]:::db
    AnalyticsS --> Analytics_DB[(analytics_db)]:::db
    UserS --> User_DB[(user_db)]:::db
    NotificationS --> Notification_DB[(notification_db)]:::db

    URLs -.→ Redis[(Redis Cache)]:::db
    Gateway -.→ Redis

    URLs -.→ RabbitMQ{RabbitMQ Broker}:::broker
    AnalyticsS -.→ RabbitMQ
    NotificationS -.→ RabbitMQ

    Prometheus[Prometheus]:::monitor -->|Scrape /metrics| Gateway
    Prometheus -->|Scrape /metrics| URLs
    Prometheus -->|Scrape /metrics| AnalyticsS
    Prometheus -->|Scrape /metrics| UserS
    Prometheus -->|Scrape /metrics| NotificationS

    Grafana[Grafana]:::monitor -->|Query| Prometheus
    Grafana -->|Query| Loki[Loki]:::monitor
```



```
Promtail[Promtail] :: monitor → |Push Logs| Loki
DockerLogs[(Docker Logs)] → Promtail
```

Mỗi microservice có cơ sở dữ liệu riêng (database-per-service pattern), giao tiếp đồng bộ qua HTTP (qua gateway) và bất đồng bộ qua RabbitMQ (message queue pattern). Redis đóng vai trò cache cho URL shortener. Gateway đóng vai trò API gateway duy nhất, chịu trách nhiệm xác thực JWT, rate limiting, circuit breaker, và thu thập metrics.

2. Docker Compose & Dockerfiles — Phát Triển & Build

File `docker-compose.yml` định nghĩa 18 containers chạy trong một network bridge duy nhất mang tên `url-shortener`.

2.1. Cấu Hình Chi Tiết Các Container

Dưới đây là bảng tổng hợp danh sách các container, image tương ứng, cấu hình cổng (port mapping) và cơ chế kiểm tra sức khỏe (healthcheck):

Container	Image & Tag	Cổng (Host:Container)	Chức Năng Chính & Cơ Chế Healthcheck
url_db	postgres:16-alpine	5432:5432	DB URL Service. Healthcheck: <code>pg_isready</code> (5s interval, 10 retries)
analytics_db	postgres:16-alpine	5433:5432	DB Analytics Service. Healthcheck: <code>pg_isready</code>
user_db	postgres:16-alpine	5434:5432	DB User Service. Healthcheck: <code>pg_isready</code>
notification_db	postgres:16-alpine	5435:5432	DB Notification Service. Healthcheck: <code>pg_isready</code>
adminer	adminer:latest	8090:8080	GUI quản lý database. Không có healthcheck
rabbitmq	rabbitmq:3.13-management-alpine	5672:5672 , 15672:15672	Message broker. Healthcheck: <code>rabbitmq-</code>

Container	Image & Tag	Cổng (Host:Container)	Chức Năng Chính & Cơ Chế Healthcheck
			diagnostics ping (10s interval)
redis	redis:7-alpine	6379:6379	Ephemeral cache (--save "" -- appendonly no). Healthcheck: redis-cli ping
nginx	nginx:1.27-alpine	80:80	Reverse proxy routing. Không có healthcheck
url-service	Build local (Go)	8081:8080	Microservice URL. Healthcheck: wget / health (10s interval, 5 retries)
analytics- service	Build local (Go)	8082:8080	Microservice Analytics. Healthcheck: wget / health
user-service	Build local (Go)	8083:8080	Microservice User. Healthcheck: wget / health
notification- service	Build local (Go)	8084:8080	Microservice Notification. Healthcheck: wget / health
gateway	Build local (Go)	8080:8080	API Gateway chính. Healthcheck: wget / health
frontend	Build local (Node)	5173:5173	React/Vite dev server. Không có healthcheck
prometheus	prom/ prometheus:v2.53.0	9090:9090	Thu thập metrics. Không có healthcheck
grafana	grafana/ grafana:11.1.0	3000:3000	Trực quan hóa dashboard. Không có healthcheck
loki	grafana/loki:2.9.1	3100:3100	Tập hợp logs. Không có healthcheck
promtail	grafana/ promtail:latest	N/A (internal: 9080)	

Container	Image & Tag	Cổng (Host:Container)	Chức Năng Chính & Cơ Chế Healthcheck
			Thu thập logs từ host/ socket. Không có healthcheck

2.2. Tài Nguyên, Phụ Thuộc và Môi Trường

- **Volumes & Mounts:** Định nghĩa 8 named volumes cho việc lưu trữ dữ liệu (DBs, RabbitMQ, Redis, Prometheus, Grafana). Sử dụng bind mounts ở chế độ chỉ đọc (ro) cho các file cấu hình (nginx.conf, prometheus.yml, loki-config.yml, promtail-config.yml) và Docker socket (/var/run/docker.sock cho Promtail).
- **Ràng Buộc Khởi Động:** Thứ tự khởi động sử dụng điều kiện service_healthy: Databases/Cache/Broker (Phase 0) $\rightarrow$ Microservices (Phase 1) $\rightarrow$ Gateway (Phase 2) $\rightarrow$ Frontend/Nginx (Phase 3).
- **Biến Môi Trường:**
 - *Databases/Broker:* Cấu hình thông tin đăng nhập mặc định (guest/guest cho RabbitMQ, admin/admin cho Grafana, credentials riêng cho từng PostgreSQL).
 - *Go Services & Gateway:* Nhận cấu hình kết nối qua DATABASE_URL, REDIS_URL, RABBITMQ_URL và các cấu hình bảo mật JWT_SECRET, IP_HASH_SALT.
 - **Restart Policies:** Chỉ có Nginx và stack monitoring được gán chính sách unless-stopped. Các microservices và databases mặc định là no (không tự khởi động lại).

2.3. Quy trình Build Docker (Dockerfiles)

Các thành phần trong hệ thống được đóng gói thông qua Dockerfile riêng biệt: - **Go Services & Gateway:** Quy trình build 2 giai đoạn (multi-stage build): - *Giai đoạn 1 (Build):* Sử dụng golang:1.23-alpine, copy toàn bộ mã nguồn (COPY . .), và biên dịch bằng go build -o main . - *Giai đoạn 2 (Runtime):* Sử dụng alpine:latest, chỉ sao chép file binary đã biên dịch từ Giai đoạn 1 nhằm giảm dung lượng ảnh (~15 MB). - *Khuyến nghị tối ưu:* Cấu hình build tĩnh (CGO_ENABLED=0), nén binary (-ldflags="-s -w"), thêm ca-certificates cho SSL, sao chép go.mod / go.sum trước để tận dụng Docker layer cache, và thêm .dockerignore. - **Frontend React/Vite:** - *Môi trường phát triển:* Build 1 giai đoạn trên node:22-alpine chạy dev mode (npm run dev) để hỗ trợ hot-reload. - *Khuyến nghị Production:* Đóng gói multi-stage (Giai đoạn 1 build code static, Giai đoạn 2 sử dụng Nginx alpine phục vụ static assets). - **Phân tích kỹ thuật build:** Quy trình build sử dụng Go Workspace (go.work) để liên kết các module cục bộ (gateway, services/*, shared/*). Cần bổ sung file .dockerignore tại thư mục root của dự án để tránh copy thừa dữ liệu không liên quan (như node_modules, k8s manifests, logs) gây invalidate cache.

3. Kubernetes Manifests

Hệ thống được cấu trúc hóa để chạy trên Kubernetes thông qua các tài nguyên được chia nhóm.

3.1. Namespace

Toàn bộ tài nguyên được cô lập trong namespace `url-shortener` để tránh xung đột với các ứng dụng khác trong cluster.

3.2. ConfigMap và Secrets

- **ConfigMap `app-config`** : Lưu trữ các cấu hình tĩnh và phi nhạy cảm, bao gồm các URL dịch vụ nội bộ (ví dụ: `http://url-service:8080`), cổng hoạt động, và chuỗi kết nối database có vô hiệu hóa SSL (`sslmode=disable`).
- **Secret `app-secrets`** : Lưu trữ khóa `JWT_SECRET`.
- **Khuyến nghị bảo mật**: Các chuỗi kết nối PostgreSQL và RabbitMQ hiện đang nằm trong ConfigMap cần được di chuyển hoàn toàn sang Secrets vì chúng chứa thông tin xác thực (username/password).

3.3. Infrastructure Deployments

Phần hạ tầng bao gồm Redis, RabbitMQ và 4 database PostgreSQL được định nghĩa dưới dạng các Kubernetes `Deployment` kết hợp với `Service` nội bộ (`ClusterIP`).

Dịch Vụ	Loại Service	Cổng Khai Báo
<code>redis</code>	ClusterIP	6379
<code>rabbitmq</code>	ClusterIP	5672 (AMQP), 15672 (Management UI)
Databases (x4)	ClusterIP	5432

Việc sử dụng tài nguyên loại `Deployment` đi kèm lưu trữ dạng `emptyDir` cho PostgreSQL là một rủi ro cực lớn. Khi các Pod bị tắt hoặc lên lịch lại (rescheduled), toàn bộ dữ liệu ghi nhận trong database sẽ bị xóa sạch. Databases trong môi trường thực tế bắt buộc phải sử dụng `StatefulSet` kết hợp với `PersistentVolumeClaim` (PVC) như phân tích ở mục 3.6.

3.4. Application Deployments

Khai báo tài nguyên cho các ứng dụng nghiệp vụ:

Tên Service	Số Lượng Pod (Replicas)	Image Pull Policy	Cách Expose
url-service	3	IfNotPresent	ClusterIP
analytics-service	1	IfNotPresent	ClusterIP
user-service	1	IfNotPresent	ClusterIP
notification-service	1	IfNotPresent	ClusterIP
gateway	2	IfNotPresent	NodePort (Cổng host: 30080)

API Gateway là điểm tiếp nhận duy nhất từ bên ngoài qua NodePort 30080, từ đó định tuyến lưu lượng đến các service nội bộ khác.

3.5. Services và NodePort

Mô hình mạng thiết lập toàn bộ các service phụ trợ và microservice chạy ẩn dưới dạng ClusterIP. Chỉ có API Gateway được cấu hình NodePort để mở luồng traffic từ bên ngoài đi vào hệ thống.

3.6. Phân Tích Deployment vs StatefulSet

Hạ tầng hiện tại cần được nâng cấp cho môi trường sản xuất theo các tiêu chuẩn sau: - **Stateless Services:** (Microservices & Gateway) tiếp tục duy trì dạng Deployment để dễ dàng nâng scale và roll-out. - **Stateful Services:** (PostgreSQL, RabbitMQ) cần chuyển đổi sang StatefulSet để đảm bảo tính định danh mạng cố định (ví dụ: url-db-0) và liên kết chặt chẽ với các volume lưu trữ vật lý độc lập thông qua volumeClaimTemplates.

4. Health Checks — Liveness và Readiness

4.1. ReadinessProbe trong Docker Compose

Docker Compose sử dụng thẻ healthcheck để giám sát sức khỏe container. Nếu lệnh kiểm tra trả về mã lỗi khác 0, container sẽ mang trạng thái "unhealthy", giúp trì hoãn các tiến trình phụ thuộc khởi động sau. Tuy nhiên, nó không tự động restart container bị lỗi trừ phi được cấu hình đi kèm chính sách restart cụ thể.

4.2. ReadinessProbe trong Kubernetes

Kubernetes sử dụng `readinessProbe` để kiểm tra khả năng sẵn sàng nhận tải của Pod. Nếu probe thất bại, Pod sẽ bị gỡ bỏ khỏi danh sách endpoint của Service, ngắt toàn bộ traffic đi vào nó. -

Microservices & Gateway: Thực hiện HTTP GET đến endpoint `/health` trên cổng `8080`. -

Redis / RabbitMQ / Postgres: Sử dụng các lệnh kiểm tra nội bộ thông qua cơ chế `exec` (`redis-cli ping`, `rabbitmq-diagnostics ping`, và `pg_isready`).

4.3. So Sánh và Đối Chiếu

Đặc Điểm	Docker Compose Healthcheck	Kubernetes ReadinessProbe
Mục Tiêu	Xác định trạng thái container	Định tuyến traffic, ngắt traffic khi lỗi
Phản Ứng	Ghi nhận trạng thái	Gỡ Pod khỏi Service Routing
Tự Phục Hồi	Không (cần restart policy hỗ trợ)	Chỉ thực hiện khi có thêm LivenessProbe

Hệ thống hiện tại trên Kubernetes hoàn toàn thiếu vắng `LivenessProbe` (để tự động khởi động lại Pod khi bị treo/deadlock) và `startupProbe` (giúp bảo vệ các service khởi động chậm như RabbitMQ khỏi bị kill sớm trong quá trình init). Cần bổ sung cả hai loại probe này cho môi trường production.

5. Infrastructure Components

5.1. PostgreSQL (4 Instances)

- **Đặc điểm:** Triển khai độc lập cho từng service, sử dụng phiên bản `postgres:16-alpine`.
- **Kết nối:** Chuỗi kết nối nội bộ dạng `postgres://[user]:[pass]@[db-host]:5432/[db-name]?sslmode=disable`.
- **Khuyến nghị Production:** Kích hoạt mã hóa SSL (`sslmode=require`), bổ sung PgBouncer để quản lý pool kết nối, thiết lập cơ chế sao lưu định kỳ (WAL archiving/pg_dump), và cấu hình Replication (Master-Slave) để dự phòng thảm họa.

5.2. Redis Cache

- **Đặc điểm:** Dùng làm cache phân tán cho URL lookups và lưu bộ đếm rate limit.
- **Chế độ chạy:** Chạy lưu hoàn toàn trên RAM (ephemeral mode: `--save "" --appendonly no`) để tối ưu hiệu năng.

- **Khuyến nghị Production:** Cấu hình xác thực mật khẩu, giới hạn dung lượng RAM tối đa (`maxmemory`), và triển khai mô hình Redis Sentinel hoặc Redis Cluster để đảm bảo tính sẵn sàng cao.

5.3. RabbitMQ Message Broker

- **Đặc điểm:** Làm trung gian truyền thông điệp bất đồng bộ giữa các service (`url-service` đẩy sự kiện, `analytics-service` và `notification-service` tiêu thụ).
- **Các Queue chính:** `url.created`, `url.accessed`, `url.deleted`, `user.registered`, `notification.send`.
- **Khuyến nghị Production:** Thay đổi tài khoản mặc định `guest`, cấu hình kết nối bảo mật AMQPS (SSL/TLS), và sử dụng Quorum Queues để bảo toàn thông điệp khi xảy ra lỗi node.

5.4. Nginx Reverse Proxy

Nginx đóng vai trò là cổng định tuyến ở mức ứng dụng ngoài cùng trong file compose:

Đường Dẫn (Path)	Dịch Vụ Đích (Upstream)	Vai Trò
<code>/api/</code>	<code>gateway:8080</code>	Chuyển tiếp các cuộc gọi API
<code>/r/</code>	<code>gateway:8080</code>	Định tuyến các request redirect URL ngắn
<code>/health</code>	<code>gateway:8080</code>	API check health của Gateway
<code>/</code>	<code>frontend:5173</code>	Phục vụ mã nguồn Frontend

- **Khuyến nghị:** Cấu hình mã hóa SSL/TLS, thêm các header bảo mật (`HSTS`, `X-Frame-Options`), kích hoạt nén `gzip`, và cài đặt cấu hình giới hạn tần suất request (rate limiting) ngay tại lớp Nginx.

5.5. Adminer

GUI quản trị dữ liệu hoạt động trên cổng 8090 . - **Khuyến nghị:** Gỡ bỏ hoàn toàn container này khỏi sơ đồ triển khai Production để tránh rò rỉ dữ liệu.

6. CI/CD Pipeline — GitHub Actions

6.1. Continuous Integration (CI)

Quy trình CI tự động chạy khi có Push hoặc Pull Request vào nhánh `main`: 1. `go-lint-and-test`: Kiểm tra định dạng code (`gofmt`), chạy phân tích tĩnh (`go vet`), và thực hiện unit test có bật tính năng phát hiện race condition (`go test -race`). 2. `frontend-build`: Cài đặt dependencies bằng lệnh tối ưu `npm ci` và build kiểm thử frontend. 3. `docker-compose-check`: Chạy lệnh build thử toàn bộ các Dockerfile để đảm bảo không bị lỗi cú pháp cấu hình.

6.2. Continuous Delivery (CD)

Kích hoạt khi push code lên `main` hoặc tạo tag dạng `v*`. Tiến hành build các image docker và đẩy (push) lên GitHub Container Registry (GHCR).

6.3. Build Matrix

Để tối ưu thời gian chạy, CD sử dụng một ma trận build (build matrix) chạy song song 6 jobs tương ứng với 6 thành phần:

Thành Phần	Thư Mục Gốc (Context)	Đường Dẫn Dockerfile
<code>url-service</code>	<code>.</code>	<code>services/url-service/Dockerfile</code>
<code>analytics-service</code>	<code>.</code>	<code>services/analytics-service/Dockerfile</code>
<code>user-service</code>	<code>.</code>	<code>services/user-service/Dockerfile</code>
<code>notification-service</code>	<code>.</code>	<code>services/notification-service/Dockerfile</code>
<code>gateway</code>	<code>.</code>	<code>gateway/Dockerfile</code>
<code>frontend</code>	<code>.</code>	<code>frontend/Dockerfile</code>

6.4. Docker Layer Caching

CD cấu hình cache thông qua GitHub Actions cache backend (`type=gha` , `mode=max`). Cơ chế này giúp giảm thời gian build các image từ 3-5 phút xuống còn dưới 60 giây ở các lần chạy sau.

6.5. Multi-Arch Build

- **Hiện tại:** Chỉ build cho kiến trúc `linux/amd64`.
- **Khuyến nghị:** Tích hợp `docker/setup-qemu-action` để hỗ trợ build thêm kiến trúc `linux/arm64` phục vụ các máy chủ tối ưu chi phí như AWS Graviton hoặc Azure ARM VMs.

6.6. Deploy to AKS

Tiến trình deploy tự động lên Azure Kubernetes Service (AKS) gồm các bước: 1. Đăng nhập Azure thông qua Service Principal credentials lưu trong GitHub Secrets. 2. Thiết lập cấu hình kết nối context AKS và cài đặt `kubectl`. 3. Tạo và cập nhật Secret truy cập Registry (`ghcr-secret`) vào namespace `url-shortener`. 4. Thay thế tag ảnh tĩnh trong file `k8s/apps.yaml` thành tag dạng động chứa commit SHA (`${{ github.sha }}`) bằng lệnh `sed`. 5. Apply các file manifest theo thứ tự: `config.yaml` -> `infra.yaml` -> `apps.yaml`. 6. Kiểm tra và xác nhận trạng thái triển khai thành công (`kubectl rollout status`) với thời gian chờ tối đa 2 phút.

7. Monitoring Stack

7.1. Prometheus — Cấu Hình Scrape

Prometheus hoạt động ở chế độ thu thập dữ liệu chủ động (pull-based) với chu kỳ rất ngắn (`scrape_interval: 5s` và `evaluation_interval: 5s`). Các endpoint thu thập bao gồm `/metrics` của cả 5 service backend (gateway và 4 microservices). - **Khuyến nghị:** Điều chỉnh retention time (`storage.tsdb.retention.time`) từ 1h (môi trường dev) lên 30d ở production, đồng thời cấu hình thêm cảnh báo (Alertmanager).

7.2. Grafana — Dashboards

Grafana được thiết lập sẵn hai dashboard thông qua cơ chế tự động nạp cấu hình (provisioning): 1. **Services Overview:** Theo dõi tài nguyên hệ thống (số lượng Goroutines, bộ nhớ Heap tiêu thụ, tỷ lệ CPU, số file descriptor đang mở) và tích hợp log stream từ Loki. 2. **Circuit Breaker Monitor:** Theo dõi trạng thái của Circuit Breaker ở Gateway (0: CLOSED, 1: HALF_OPEN, 2: OPEN), số lượng yêu cầu bị từ chối, tần suất lỗi, và biểu đồ trễ p50/p95/p99.

7.3. Loki — Log Aggregation

Loki thu thập và nén log dạng phân tán. Cấu hình sử dụng động cơ lưu trữ TSDB (`schema: v13`) ghi trực tiếp lên ổ đĩa của container, và tắt chế độ xác thực (`auth_enabled: false`) cho môi trường dev.

7.4. Promtail — Log Collector

Promtail chạy dưới dạng một daemon thu thập log từ Docker socket (`/var/run/docker.sock`) và thư mục chứa log container của máy chủ (`/var/lib/docker/containers`). Nó tự động phân tích và gán nhãn log dựa trên compose metadata để đẩy về Loki.

7.5. Datasource Provisioning

Các nguồn dữ liệu Prometheus (mặc định) và Loki được tự động đăng ký vào Grafana khi container khởi chạy và bị khóa quyền chỉnh sửa từ giao diện người dùng (`editable: false`) để tránh sai lệch cấu hình.

8. Logging Stack

8.1. Loki Configuration

Loki chạy ở cấu hình Single Instance (`replication_factor: 1` , lưu trữ in-memory ring index). Các dữ liệu log mẫu cũ hơn 7 ngày sẽ bị từ chối nhận (`reject_old_samples_max_age: 168h`). Dữ liệu log được lưu trữ tại thư mục `/tmp/loki` nên mang tính tạm thời và sẽ biến mất khi container bị khởi động lại.

8.2. Promtail Configuration

Promtail thực hiện quét Docker socket định kỳ mỗi 5s để nhận diện container mới. Nó chuyển đổi nhãn container nội bộ của Docker Compose thành các tag nhãn tường minh như `service` (tên dịch vụ) và `container` (tên container cụ thể) giúp dễ dàng truy vấn log. Vị trí dòng log cuối cùng đã đọc được lưu lại trong file `/tmp/positions.yaml` để tránh gửi trùng lặp log.

8.3. Grafana Log Explorer

Log được hiển thị trực tiếp trong dashboard tổng quan của Grafana. Người dùng có thể lọc nhanh log bằng biến môi trường `$service` được lấy động từ Loki. Panel hỗ trợ các tính năng như xem log theo thời gian thực (live tailing), tự động xuống dòng, và hiển thị cấu trúc JSON log đẹp mắt.

9. Container Dependency Graph

9.1. Dependency Tree

Quan hệ phụ thuộc giữa các thành phần được thể hiện qua sơ đồ kiến trúc dưới đây:

```
graph TD
    classDef db fill:#85C1E9,stroke:#2C3E50,stroke-width:2px;
    classDef svc fill:#82E0AA,stroke:#2C3E50,stroke-width:2px;
    classDef monitor fill:#D7BDE2,stroke:#2C3E50,stroke-width:2px;

    %% Level 0: Databases
    url_db[url_db] ::: db
    analytics_db[analytics_db] ::: db
    user_db[user_db] ::: db
    notification_db[notification_db] ::: db
    redis[redis] ::: db
    rabbitmq[rabbitmq] ::: db

    %% Level 1: Microservices
    url-service[url-service] ::: svc
    analytics-service[analytics-service] ::: svc
    user-service[user-service] ::: svc
    notification-service[notification-service] ::: svc

    %% Level 2: Gateway
    gateway[gateway] ::: svc

    %% Level 3: Frontend & Proxy
    nginx[nginx]
    frontend[frontend]

    %% Level 4: Monitoring
    prometheus[prometheus] ::: monitor
    grafana[grafana] ::: monitor
    loki[loki] ::: monitor
    promtail[promtail] ::: monitor

    %% Dependencies
    url-service --> url_db
    url-service --> redis
    url-service --> rabbitmq

    analytics-service --> analytics_db
    analytics-service --> rabbitmq
```

```

user-service → user_db

notification-service → notification_db
notification-service → rabbitmq

gateway → url-service
gateway → analytics-service
gateway → user-service
gateway → notification-service

nginx → gateway
nginx → frontend
frontend → gateway

prometheus → gateway
grafana → prometheus
grafana → loki
promtail → loki

```

9.2. Startup Order

Trình tự khởi động tuần tự qua các giai đoạn được biểu diễn như sau:

```

flowchart TD
    subgraph Phase0 ["Phase 0: Infrastructure (Parallel)"]
        db[PostgreSQL Databases]
        red[Redis Cache]
        rab[RabbitMQ Broker]
    end

    subgraph Phase1 ["Phase 1: Microservices (Parallel)"]
        svcs[Microservices: URL, Analytics, User, Notification]
    end

    subgraph Phase2 ["Phase 2: Gateway"]
        gw[API Gateway]
    end

    subgraph Phase3 ["Phase 3: Frontend & Proxy"]
        web[Frontend & Nginx Proxy]
    end

    subgraph Phase4 ["Phase 4: Monitoring (Independent)"]
        mon[Prometheus, Grafana, Loki, Promtail]
    end

    Phase0 -->|Healthcheck Passes| Phase1
    Phase1 -->|Healthcheck Passes| Phase2
    Phase2 -->|Healthcheck Passes| Phase3
    Phase1 -.→ Phase4
    Phase2 -.→ Phase4

```

9.3. Critical Path Analysis

Tuyến Khởi Động Trọng Yếu (Critical Path):

```
rabbitmq (Chờ 20s) → url-service (Chờ 15s) → gateway (Chờ 20s) → nginx
```

Tổng thời gian để toàn bộ hệ thống ở trạng thái sẵn sàng tiếp nhận request rơi vào khoảng **70 đến 100 giây**. - **Điểm nghẽn chính:** RabbitMQ mất nhiều thời gian nhất để khởi tạo và chạy các script diagnostics. - **Điểm yếu hệ thống (SPOF):** RabbitMQ, Redis, Nginx, và các database PostgreSQL chỉ chạy duy nhất 1 instance. Nếu bất kỳ thành phần nào trong số này gặp sự cố, một phần hoặc toàn bộ hệ thống sẽ ngừng hoạt động.

10. Production Deployment Recommendations

10.1. Ingress Controller

Thay thế Nginx container bằng một Kubernetes Ingress Controller (ví dụ: Nginx Ingress) để quản lý traffic. Điều này giúp tích hợp giải pháp tự động cấp phát SSL/TLS (Cert-Manager với Let's Encrypt), định tuyến đường dẫn mềm dẻo, và thực hiện cân bằng tải tốt hơn ở lớp ngoài cùng của cluster.

10.2. Horizontal Pod Autoscaler (HPA)

Cần cấu hình tự động co giãn số lượng Pod dựa trên mức độ sử dụng tài nguyên thực tế:

Dịch Vụ	Số Lượng Pod Min	Số Lượng Pod Max	Ngưỡng Kích Hoạt CPU	Ngưỡng Kích Hoạt Memory	Chỉ Số Đo Lường Khác
url-service	3	20	70%	80%	> 1000 http_req/sec
analytics-service	1	5	70%	80%	N/A
user-service	1	5	70%	80%	N/A
notification-service	1	5	70%	80%	N/A
gateway	2	10	70%	80%	> 5000 http_req/sec

10.3. StatefulSet cho Databases

Chuyển đổi toàn bộ các database PostgreSQL từ Deployment sang StatefulSet đi kèm với cấu hình PersistentVolumeClaim (PVC) để lưu trữ dữ liệu bền vững trên các Premium SSD (như managed-premium trên Azure). Thiết lập thời gian chờ tắt Pod (terminationGracePeriodSeconds) là 30s để tránh ngắt đột ngột các transaction đang ghi dở như phân tích ở mục 3.6.

10.4. Resource Requests và Limits

Bắt buộc phải thiết lập hạn mức tài nguyên (Resource Quota) cho từng Pod để tránh tình trạng tranh chấp tài nguyên (Noisy Neighbor) trong cluster:

Pod	CPU Request	Memory Request	CPU Limit	Memory Limit
url-service	200m	256Mi	1.0	1Gi
analytics-service	100m	128Mi	500m	512Mi
user-service	100m	128Mi	500m	512Mi
notification-service	100m	128Mi	500m	512Mi
gateway	200m	256Mi	1.0	1Gi
redis	100m	128Mi	500m	512Mi
rabbitmq	200m	512Mi	1.0	2Gi
postgres (x4)	250m	512Mi	2.0	4Gi
prometheus	500m	1Gi	2.0	4Gi
loki	500m	1Gi	2.0	4Gi

10.5. PersistentVolumeClaims

Cần cấp phát dung lượng ổ đĩa vật lý bền vững cho các thành phần lưu trữ: - **Prometheus**: Cấp phát 50 GB. - **Grafana**: Cấp phát 10 GB. - **Loki**: Cấp phát 100 GB. - **Dự toán lưu trữ trong 30 ngày**: Ước tính hệ thống tiêu tốn khoảng **205 GB** dung lượng đĩa Premium SSD cho toàn bộ database, log và metrics.

10.6. Network Policies

Thiết lập NetworkPolicies để phân đoạn mạng nội bộ cluster, ngăn chặn các truy cập trái phép chéo giữa các tầng:

Lớp (Tier)	Nhãn Nhận Diện (Labels)	Cho Phép Nhận Traffic Từ (Ingress)	Quyền Gửi Traffic Đi (Egress)
Database	<code>tier: database</code>	Chỉ từ <code>tier: backend</code> (Cổng 5432)	Không cho phép ra ngoài
Backend	<code>tier: backend</code>	Chỉ từ <code>app: gateway</code> (Cổng 8080)	Gửi đến Database, Redis, RabbitMQ
Gateway	<code>app: gateway</code>	Chỉ từ Ingress Controller	Gửi đến <code>tier: backend</code>
Frontend	<code>tier: frontend</code>	Chỉ từ Ingress Controller	Không có

10.7. Pod Disruption Budgets

Thiết lập chính sách PDB để đảm bảo số lượng Pod tối thiểu luôn hoạt động khi cluster tiến hành nâng cấp hoặc bảo trì định kỳ: - `url-service-pdb: minAvailable: 2` (Luôn duy trì ít nhất 2 Pod hoạt động). - `gateway-pdb: minAvailable: 1`.

10.8. Secret Management

Sử dụng giải pháp quản lý khóa tập trung (như Azure Key Vault kết hợp với Secret Provider Class CSI Driver) thay vì lưu trực tiếp khóa Base64 trong các file YAML. Pod sẽ lấy quyền truy cập thông qua cơ chế AAD Pod Identity hoặc Managed Identity một cách an toàn.

11. Kết Luận

Tổng Kết Kiến Trúc

Hệ thống URL Shortener Microservices đã xây dựng được một nền tảng triển khai microservices bài bản: tách biệt database cho từng dịch vụ, hỗ trợ đầy đủ các cơ chế kiểm tra sức khỏe và ràng buộc thứ tự khởi động, tích hợp sẵn pipeline CI/CD tự động hóa, cùng hệ thống thu thập log/metrics toàn diện qua Grafana.

Điểm Mạnh và Điểm Yếu

Hệ thống có ưu điểm lớn ở tính cô lập dịch vụ (database-per-service), khả năng giám sát trạng thái chi tiết, và quy trình CI/CD hoàn thiện. Tuy nhiên, kiến trúc triển khai hiện tại vẫn còn nhiều lỗ hổng lớn cần khắc phục trước khi đưa vào vận hành thực tế:

Vấn Đề	Mức Độ	Giải Pháp
Mất dữ liệu database khi Pod restart	CRITICAL	Chuyển PostgreSQL sang <code>StatefulSet</code> + PVC như phân tích mục 3.6
Lộ mật khẩu trong ConfigMap	HIGH	Di chuyển toàn bộ thông tin nhạy cảm sang K8s Secrets hoặc Vault
Không giới hạn tài nguyên Pod	HIGH	Bổ sung khai báo <code>requests</code> và <code>limits</code> cho mọi Pod
Hạ tầng database/broker chỉ có 1 node	HIGH	Triển khai mô hình Cluster cho PostgreSQL, Redis, RabbitMQ
Thiếu cơ chế tự phục hồi Pod khi treo	MEDIUM	Bổ sung <code>LivenessProbe</code> và <code>startupProbe</code>

Security Checklist cho Production

- [] Thay đổi toàn bộ thông tin xác thực mặc định (đổi pass `guest` , pass admin Grafana).
- [] Mã hóa toàn bộ dữ liệu truyền nhận nội bộ cluster bằng SSL/TLS.
- [] Chuyển các key và pass sang Azure Key Vault.
- [] Áp dụng NetworkPolicies để cô lập cơ sở dữ liệu.
- [] Quét lỗ hổng bảo mật của các Docker Image (`Trivy` hoặc `Snyk`) trong pipeline CI/CD.

Dự Toán Chi Phí Hàng Tháng (Azure AKS)

Ước tính chi phí vận hành hệ thống ở quy mô sản xuất tiêu chuẩn (3 nodes `Standard_D4s_v3` , 8 Premium SSDs, Azure Database for PostgreSQL Flexible, Azure Cache for Redis) rơi vào khoảng **\$1,340 / tháng**. Chúng ta có thể tiết kiệm chi phí bằng cách tự vận hành PostgreSQL trên Kubernetes StatefulSet thay vì sử dụng dịch vụ Managed Database của Azure.

Kiểm Thử (Testing) — url-shortener-microservices

1. Unit Tests

- **What:** Go `testing` package, table-driven tests với struct test cases, subtest qua `t.Run`
- **Where:** 6 file test trên 6 packages — url-service (base62, validation, cache, handler), gateway (circuit breaker, rate limiter, JWT, CORS), user-service (auth, bcrypt, JWT), analytics-service (click, milestone, stats), notification-service (consumer, handler), shared/events (JSON roundtrip)
- **Why:** Cô lập từng module, verify state machine chuyển tiếp (CB), roundtrip encoding (base62), và HTTP status mapping

Coverage chính

Package	File Test	Số test case	Input mẫu	Expected
base62	<code>url_test.go</code>	3 func (8+5 cases)	<code>0→"0"</code> , <code>61→"z"</code> , <code>62→"10"</code>	Encode/Decode roundtrip, lỗi invalid chars, codegen
cache	<code>url_test.go</code>	1 func (4 subtests)	miss, hit, delete, bad client fallback	Get/Set/Delete, nil fallback when Redis down
handler shorten	<code>url_test.go</code>	4 subtests	valid URL, collision retry, unauthorized, invalid URL	201 Created, 401, 400, retry logic
handler redirect	<code>url_test.go</code>	5 subtests	cache hit, cache miss → DB, deactivated, expired, not found	308, 410 (×2), 404
handler list/delete	<code>url_test.go</code>	4 subtests	success with pagination,	200, cursor, 204, 403

Package	File Test	Số test case	Input mẫu	Expected
			hasMore, delete success, forbidden	
circuitbreaker	gateway_test.go	1 func (full lifecycle)	2 failures → OPEN, reject, half-open success → CLOSED, probe fail → OPEN	State machine CLOSED → OPEN → HALF-OPEN → CLOSED/OPEN
ratelimit	gateway_test.go	1 func (2 scenarios)	deny → 429 with Retry-After, fail-open when Redis down → 201	429 / 201 Created
JWT middleware	gateway_test.go	2 func (3 scenarios)	missing token → 401, valid token → 204, public route skip	401 / 204 / next called
CORS middleware	gateway_test.go	1 func (2 scenarios)	OPTIONS preflight → 204 with headers, GET → 200 with ACAO	204 + CORS headers, 200
user-service	user_test.go	12+ func	email/password validation, bcrypt hash/verify, JWT issue/verify, register/login handlers	201/200/400/401/409

Pattern chung

flowchart LR

A[Define test cases struct] → B[Loop over cases]

B → C[t.Run subtest]

C → D[Arrange mocks]

D → E[Call SUT]

E → F[Assert status / JSON / side-effect flags]

2. Error Handling & HTTP Mapping

- **What:** Sentinel errors pattern — `var ErrX = errors.New("...")` ở mỗi service
- **Why:** So sánh bằng `errors.Is()` thay vì string matching; mapping tập trung ở handler layer

Bảng mapping error → HTTP status

Error	HTTP Status	Ý nghĩa
<code>ErrInvalidURL</code>	400 Bad Request	URL không hợp lệ
<code>ErrAlreadyExists</code>	409 Conflict	Short code đã tồn tại
<code>ErrForbidden</code>	403 Forbidden	Không phải chủ sở hữu
<code>ErrNotFound</code>	404 Not Found	Không tìm thấy URL
<code>ErrExpired</code> / <code>ErrDeactivated</code>	410 Gone	Link hết hạn hoặc bị xóa
<code>ErrDatabaseError</code>	500 Internal Server Error	Lỗi DB
<code>ErrCircuitOpen</code>	503 Service Unavailable	Circuit breaker đang OPEN
<code>ErrDuplicateEmail</code>	409 Conflict	Email đã được đăng ký
<code>ErrUserNotFound</code> / <code>ErrTokenInvalid</code>	401 Unauthorized	Sai credentials / token
<code>ErrInvalidEmail</code> / <code>ErrInvalidPassword</code>	400 Bad Request	Validation đầu vào

Kiến trúc sentinel errors

```
services/url-service/errors.go    → 8 sentinel errors
services/user-service/errors.go   → 6 sentinel errors
gateway/errors.go                 → writeError helper
gateway/circuitbreaker.go         → ErrCircuitOpen
services/analytics-service/errors.go → writeError helper
services/notification-service/errors.go → writeError helper
```

3. E2E Testing

- **What:** Shell script `e2e_test.sh` dùng `curl` + `python3` inline
- **Why:** Mô phỏng real-user flow qua gateway, kiểm tra toàn bộ chain service

Flow 11 bước

```
sequenceDiagram
    participant T as Test Script
    participant G as Gateway
    participant US as url-service
    participant AS as analytics-service
    participant NS as notification-service
    T->>G: 1. POST /api/auth/register
    T->>G: 2. POST /api/auth/login → get TOKEN
    T->>G: 3. POST /api/shorten (with TOKEN) → SHORT_CODE
    loop 15 lần
        T->>G: 4. GET /r/{SHORT_CODE}
    end
    Note over T,US: 5. sleep 5s (đợi outbox + consumer)
    T->>AS: 6. GET /api/stats/{SHORT_CODE} → assert total_clicks ≥ 15
    T->>NS: 7. GET /api/notifications → assert milestone.reached
    T->>G: 8. DELETE /api/urls/{SHORT_CODE} → 204
    T->>G: 9. GET /r/{SHORT_CODE} → 410 Gone
    loop 11 lần
        T->>G: 10. POST /api/shorten → eventually 429
    end
    T->>G: 11. GET /health → assert X-Correlation-ID header
```

Bước kiểm thử chi tiết

Bước	Endpoint	Expected	Xác minh
register	POST /api/auth/register	201	JSON response
login	POST /api/auth/login	200	Trích xuất token
shorten	POST /api/shorten	201	Trích xuất short_code
redirect (×15)	GET /r/{code}	301 / 308	HTTP status code
stats	GET /api/stats/{code}	200	total_clicks ≥ 15
notifications	GET /api/notifications	200	có milestone.reached
delete	DELETE /api/urls/{code}	204	HTTP status
deleted redirect	GET /r/{code}	410	Gone status
rate limit	POST /api/shorten ×11	429	Eventually rate limited

Bước	Endpoint	Expected	Xác minh
correlation	GET /health	200	X-Correlation-ID header

4. Load Testing (k6)

- **What:** 2 kịch bản k6 — `load_test.js` (ramping VUs) và `load_test_1m_rps.js` (constant arrival rate)
- **Why:** Đo throughput thực tế dưới tải, kiểm tra circuit breaker trip/recover

Scenario 1 — Ramping VUs (`load_test.js`)

```

flowchart LR
    subgraph Stages
        A["20s → 200 VUs"] --> B["20s → 500 VUs"]
        B --> C["20s → 1000 VUs"]
        C --> D["60s hold at 1000 VUs"]
        D --> E["20s → 0 VUs"]
    end
    subgraph Thresholds
        F["p(95) < 2000ms"]
        G["error_rate < 95%"]
    end
    Stages --> F
    Stages --> G

```

Stage	Duration	Target VUs	Mục đích
Warm-up	20s	200	Khởi tạo kết nối
Build-up	20s	500	Tăng dần tải
Peak ramp	20s	1000	Đạt ~10k req/s
Hold	60s	1000	Quan sát CB trip/recover
Ramp down	20s	0	Dừng an toàn

- **Custom metrics:** `circuit_breaker_open_responses` (Counter), `error_rate` (Rate), `redirect_duration_ms` (Trend)
- **Circuit breaker simulation:** `docker compose stop url-service` trong lúc chạy → gateway trả 503

Scenario 2 — Constant Arrival Rate (`load_test_1m_rps.js`)

- **Executor:** `constant-arrival-rate` , rate = 100.000 req/s (cấu hình qua `-e RATE=`)
- **Pre-allocated VUs:** 2.000, max VUs: 50.000
- **Mixed workload:** 30% `create` + 70% `redirect` (tỷ lệ cấu hình qua `CREATE_RATIO`)
- **URL pool:** Setup tự động tạo pool codes để tránh contention

Thresholds so sánh

Scenario	p(95)	Failure rate	Ghi chú
Ramping VUs (redirect)	< 2000ms	< 95%	Relaxed — kỳ vọng CB mở
Constant arrival (mixed)	< 10000ms	< 99%	High-throughput tolerant

Tổng kết

- **Unit test:** 6 file, ~50+ test case, coverage trải đều 6 services
- **Error mapping:** 8 sentinel errors ở url-service, mapping rõ ràng ra HTTP status
- **E2E:** 11 bước flow xuyên suốt
register → login → shorten → redirect → stats → notification → delete → rate limit
- **Load test:** Kịch bản ramping VUs lên 1000, constant arrival lên 100K req/s, verify CB behavior

Góc Nhìn Kỹ Thuật — Timing Attack, Outbox Lock & Graceful Shutdown

1. Timing Attack Mitigation — Login Endpoint

Vấn đề

Nếu handler chỉ gọi `bcrypt.Verify` khi user tồn tại, response time sẽ khác nhau giữa email hợp lệ (~200ms) và không hợp lệ (~1ms), tạo timing side-channel cho attacker dò email.

Giải pháp: Dummy Bcrypt Hash

Code hiện tại đã áp dụng dummy bcrypt — luôn gọi `Verify` với hash giả khi user không tồn tại, xoá timing difference:

```
if user == nil {
  _ = h.hasher.Verify(req.Password, dummyBcryptHash) // ~200ms
  writeError(w, http.StatusUnauthorized, "invalid credentials")
  return
}
```

```
sequenceDiagram
    autonumber
    actor Attacker
    participant H as HTTP Handler
    participant S as User Store
    participant B as bcrypt

    Attacker->>H: POST /login {email, password}
    H->>S: FindByEmail(email)

    alt Email không tồn tại
        S-->>H: nil
        H->>B: Verify(password, dummyBcryptHash) <- cost 12
        B-->>H: ~200ms elapsed
        H-->>Attacker: 401 Invalid Credentials
    else Email tồn tại
        S-->>H: User {password_hash}
        H->>B: Verify(password, realHash) <- cost 12
        B-->>H: ~200ms elapsed
    end
```

```
H-->>Attacker: 401 / 200 + Token
end
```

Phân tích

- **Ưu điểm:** Xóa timing side-channel, dùng đúng bcrypt cost, không leak email info.
- **Nhược điểm:** CPU waste — mỗi request email sai vẫn tốn ~200ms bcrypt. Trên high-traffic login, DDoS amplification risk (attacker spam email sai → server chạy bcrypt vô ích).
- **Alternative:** Dùng sleep-based delay nhẹ hơn (`time.Sleep` sau DB miss), nhưng dễ bị phát hiện qua timing pattern và không an toàn nếu goroutine scheduler delay không ổn định. Cách dùng bcrypt dummy an toàn hơn nhưng nặng CPU hơn.

Tiêu chí	Dummy bcrypt	Sleep delay
Timing consistency	Rất tốt (thuật toán cố định)	Phụ thuộc scheduler
CPU cost	Cao (~200ms/request)	Rất thấp
DDoS resilience	Kém (tốn CPU)	Tốt hơn
Implementation	Đơn giản, khó sai	Dễ sai (sleep không đủ)

2. Outbox Pattern & FOR UPDATE SKIP LOCKED

Vấn đề

URL Service dùng transactional outbox để đảm bảo at-least-once delivery. Nhiều replica chạy `OutboxCoordinator.Run()` đồng thời, cùng quét bảng `outbox` → **duplicate processing, double-send nếu không có cơ chế lock.**

Giải pháp: CTE + FOR UPDATE SKIP LOCKED

```
WITH claimed AS (
  SELECT id
  FROM outbox
  WHERE published_at IS NULL
    AND (locked_until IS NULL OR locked_until < now())
  ORDER BY created_at ASC
  LIMIT $1
  FOR UPDATE SKIP LOCKED
)
UPDATE outbox o
SET locked_until = now() + interval '30 seconds'
```



```
FROM claimed
WHERE o.id = claimed.id
RETURNING o.id, o.event_type, o.payload, o.created_at, o.locked_until, o.published_at
```

Thành phần	Vai trò
WITH claimed ... FOR UPDATE SKIP LOCKED	Chọn bản ghi chưa published và không bị replica khác lock , bỏ qua bản ghi đang xử lý
UPDATE ... SET locked_until = now() + 30s	Giành quyền xử lý, các replica khác sẽ bỏ qua trong 30s
RETURNING	Trả về dữ liệu ngay, tránh SELECT riêng

```
flowchart TD
    subgraph Poller [OutboxCoordinator - Poll every 2s]
        Start([Tick]) --> CTE["CTE Query: FOR UPDATE SKIP LOCKED<br/>>(batch_size=50)"]
        CTE --> Lock["SET locked_until = now() + 30s"]
        Lock --> Chan["Push to jobs channel (buffered 50)"]
    end

    subgraph Workers [3 Workers - goroutine pool]
        Chan --> W1["Worker 1"]
        Chan --> W2["Worker 2"]
        Chan --> W3["Worker 3"]

        W1 --> Pub["Publish to RabbitMQ<br/>(mutex trên channel)"]
        Pub --> Mark["UPDATE published_at = now()<br/>locked_until = NULL"]

        W2 --> Pub
        W3 --> Pub
    end
```

Thread Safety

- Worker gọi `publisher.Publish()` → bên trong dùng `sync.Mutex` trên `amqp.Channel` (RabbitMQ channel không thread-safe).
- Mỗi worker publish xong mới `MarkPublished` — nếu mark fail, record vẫn unpublished, sẽ được poll lại sau 30s.

Phân tích

Khía cạnh	Đánh giá
At-least-once	✅ Đảm bảo — nếu crash giữa publish và mark, record sẽ được retry
Exactly-once	❌ Không — consumer phải tự deduplicate bằng event ID

Khía cạnh	Đánh giá
Dead letter queue	❌ Không có — events fail nhiều lần (VD: publish lỗi liên tục) sẽ stuck mãi, không có DLQ để routing
Lock expiration	30s — nếu worker crash, record được release sau 30s; nếu publish lâu hơn 30s, replica khác có thể pick up duplicate

3. Graceful Shutdown

Vấn đề

Khi nhận SIGINT/SIGTERM, nếu shutdown đột ngột có thể gây:

- Mất event trong jobs channel chưa publish
- Transaction đang dở
- Corrupted state ở RabbitMQ/RDB

Giải pháp: Context Cascade + HTTP Drain

```
sequenceDiagram
    autonumber
    participant OS as OS Signal
    participant Main as func main()
    participant Ctx as Root Context
    participant OC as Outbox Coordinator
    participant SRV as HTTP Server
    participant DB as PostgreSQL
    participant MQ as RabbitMQ

    OS->>Main: SIGINT / SIGTERM
    Main->>Ctx: cancel() ← hủy context
    Ctx->>OC: ctx.Done() → poll & workers dừng
    Note over OC: Drain jobs channel<br/>Worker finish current publish

    Main->>SRV: Shutdown(shutdownCtx, 10s timeout)
    Note over SRV: Drain in-flight requests<br/>Reject new connections

    SRV-->>Main: OK / Timeout

    Main->>DB: defer pool.Close()
    Main->>MQ: defer rmqConn.Close()
    Note over Main: All connections closed
```

Bước	Hành động	Chi tiết
1	Nhận signal	<code>signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)</code>
2	Cancel root context	<code>cancel() → OutboxCoordinator.Run()</code> thoát vòng lặp, close jobs channel, <code>workers.Wait()</code>
3	HTTP Shutdown	<code>srv.Shutdown(shutdownCtx)</code> với timeout 10s — drain active requests
4	Đóng pool	<code>pool.Close()</code> (DB), <code>redisClient.Close()</code> , <code>rmqConn.Close()</code> qua defer

Điểm yếu trong implementation hiện tại

Service	Vấn đề
url-service	<code>srv.Shutdown</code> chạy sau <code>cancel()</code> , nhưng <code>pool.Close()</code> qua defer — nếu shutdown timeout, defer vẫn chạy nên không mất kết nối. Tuy nhiên: không chờ <code>OutboxCoordinator</code> worker finish hẳn trước khi đóng DB.
user-service	<code>srv.Shutdown(ctx)</code> dùng ctx đã bị cancel → shutdown context hết hiệu lực ngay lập tức, không có drain đúng nghĩa . Shutdown timeout = 0.
gateway	<code>srv.Shutdown(context.Background())</code> — dùng context vô hạn, tốt nhưng không có timeout.

Khuyến nghị

- Dùng `shutdownCtx` riêng với timeout (giống url-service) thay vì dùng lại ctx đã cancel.
- Chờ `OutboxCoordinator` workers hoàn thành trước khi `pool.Close()` — hiện tại chạy đồng thời qua defer, có nguy cơ DB đóng giữa lúc worker gọi `MarkPublished`.

Báo cáo phân tích từ codebase thực tế — tập trung vào 3 góc nhìn kỹ thuật có tính ứng dụng cao.
